IERG4320/IEMS5726: Data Science in Practice

Lecture 9: Case Studies (I) Business Problems

Prof. YAN Wenjing, Dr. Danny Yip
Department of Information Engineering

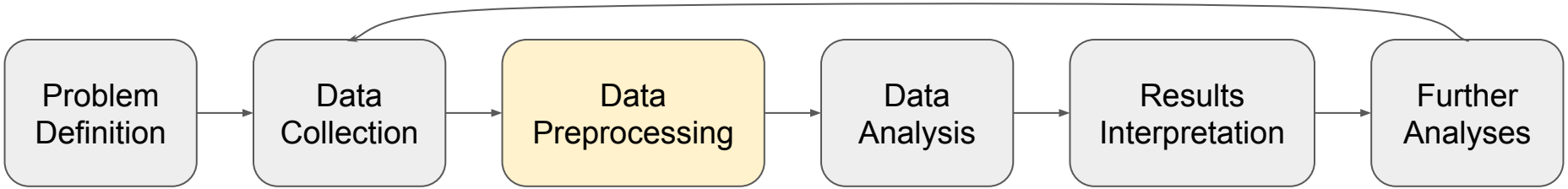
香港中文大學
The Chinese University of Hong Kong

Outline

1. Revision: Pipeline for Data Science
2. Introduction to Business Analytics
3. Recommender System
4. Stock Market Index Prediction
5. Investment Strategy

All rights are reserved to Prof. YAN Wenjing, Dr. Danny YIP,
The Department of Information Engineering, CUHK

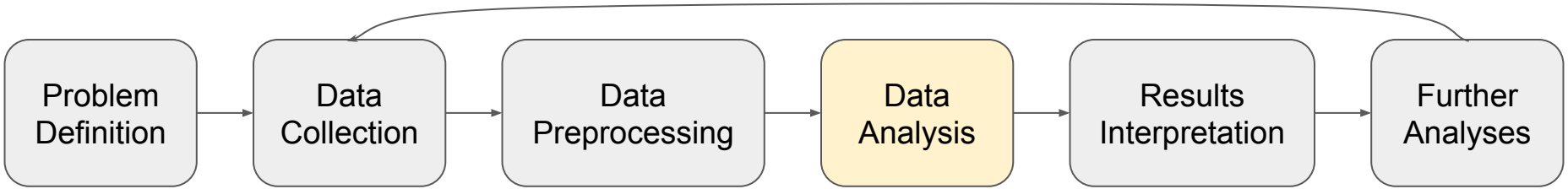
Data Preprocessing and Representation - Lecture 3 - 5



- We have learnt:

1. Data wrangling
2. Handling missing data
3. Dimension reduction and data normalization
4. Splitting datasets to training data, validation data and testing data
5. Handling numeric data, categorical data, natural language, images and audio signals with (i) computational methods and (ii) autoencoder

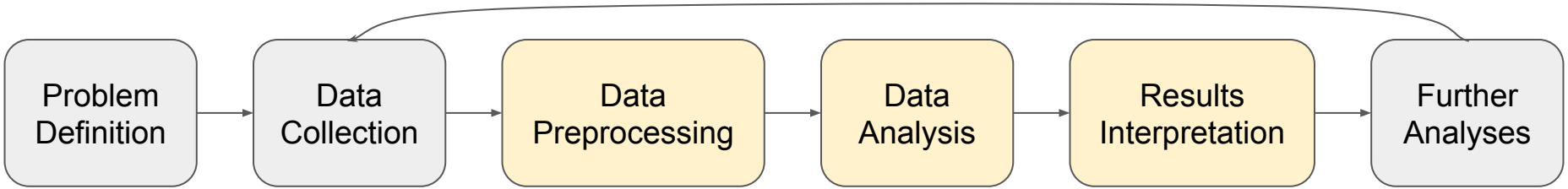
Data Modelling - Lecture 6 - 7



- We have learnt:

1. Supervised Learning: Classification and Regression
2. Unsupervised Learning: Clustering
3. Reinforcement Learning
4. (Later in text mining and multimedia) Semi-Supervised Learning: Generative Models

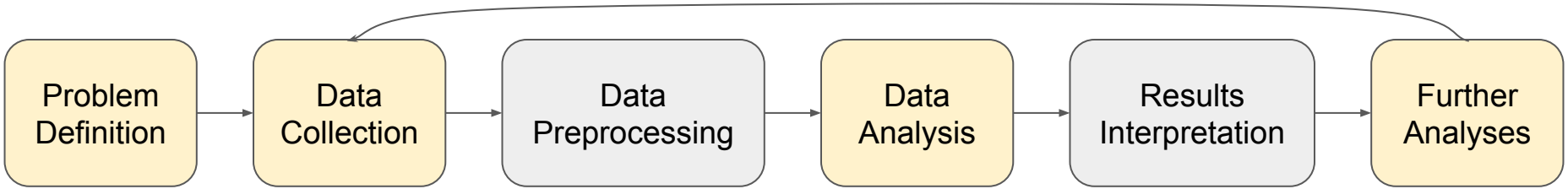
Data Visualization - Lecture 8



- We have learnt:

1. Visualize the distribution of the data
2. Visualize the performance of the data analysis
3. Visualize the data models
4. Present the findings using plots and graphs

What's Next?



- We will study complete examples using case studies:
  - Business, NLP, Multimedia, Game problems
- You can expect we will cover:

1. The importance of the problem, the problem domain specific interests
2. The challenges in collecting the data
3. More sophisticated machine learning models
4. The continuous investigation of the problem

Introduction to Business Analytics

Introduction to Business Analytics

- Business analytics is the practice of using data analysis and statistical methods to gain insights and support decision-making.

1. To improve operations,
2. To enhance customer experiences, and
3. To drive strategic planning

Introduction to Business Analytics

Features	Business Analytics	Data Science
Primary Focus	Solving specific business problems and driving decision-making	Uncovering hidden patterns and building complex, predictive models using advanced algorithms
Data	Structured data from business systems, e.g. sales, index, … etc	Both structured and unstructured data
Approaches	More business-oriented and strategic, e.g. What business decision can we make from this data?	More technical and algorithmic, e.g. What new insights can we find from this data?
Deliverables	Dashboards, reports, strategic recommendations	Predictive models, algorithms, and data products

Importance of Business Analytics

1. **Informed Decision-Making**: Replacing feeling and guessing with data-backed evidence.
2. **Improved Efficiency**: Identifying bottlenecks and optimizing operations.
3. **Increased Revenue**: Discovering new sales opportunities and improving customer experience.
4. **Reduced Risk**: Forecasting potential downturns and mitigating threats, e.g. business metrics reports.

My Understanding to Business Analytics

- Business analytics is the bridge between data and business strategy, empowering organizations to act intelligently and confidently in an increasingly complex world.
- Most data are numeric and categorical.
- The machine learning algorithms used are classic and intuitive, easier to visualize and present the data models to the client, e.g. decision tree.
- It is more about the presentation, data visualization and generating the dashboard.
- It requires the communication with both (i) the stakeholders and (ii) the IT team.
- It requires some extra skills (not covering in this course), e.g. relational database (SQL), and software methodologies (Agile and Scrum).

Recommender System

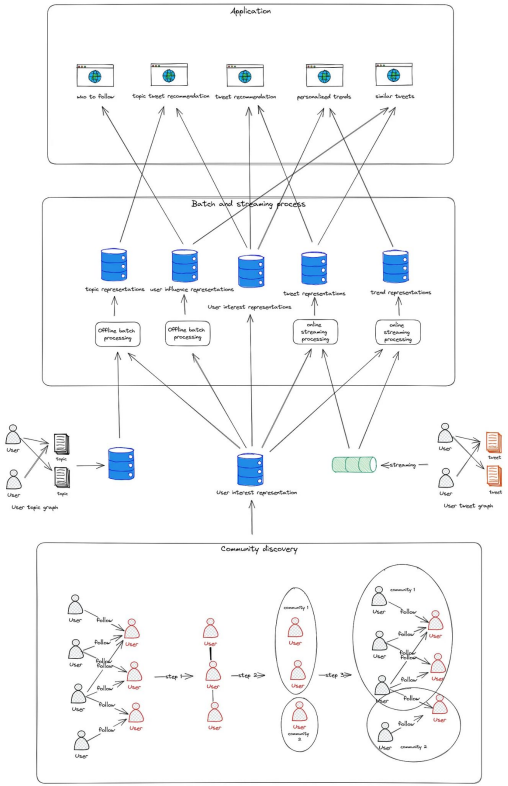


Image credit: https://happystrongcoder.substack.com/p/dive-into-twitters-recommendation-b83

Successful Recommender System - Amazon

- Amazon’s product recommendation
- Amazon suggests products based on **user behavior**, **purchase history**, and **similar users' preferences**.
- Amazon gains **average order value** and **customer satisfaction**.
- The AI-powered recommendation engine generates **approximately 35%** of Amazon's annual sales.

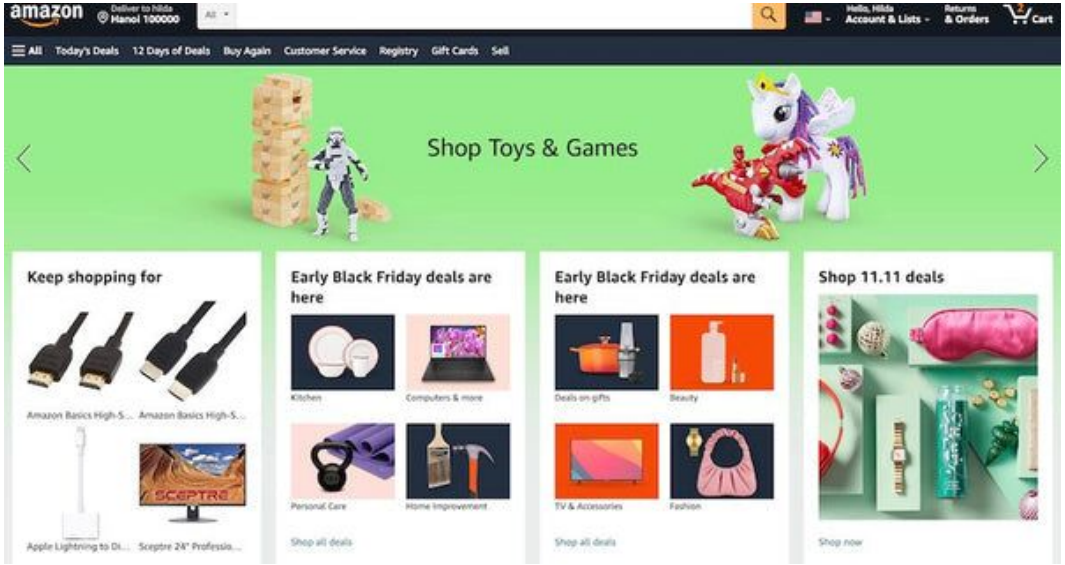


Image credit: Amazon

Successful Recommender System - Netflix

- Netflix movie recommendation
- Netflix suggests movies based on **viewing history**.
- Netflix enhances **user engagement** and **retention rates**.
- **Over 80%** of what people watch comes from these personalized suggestions

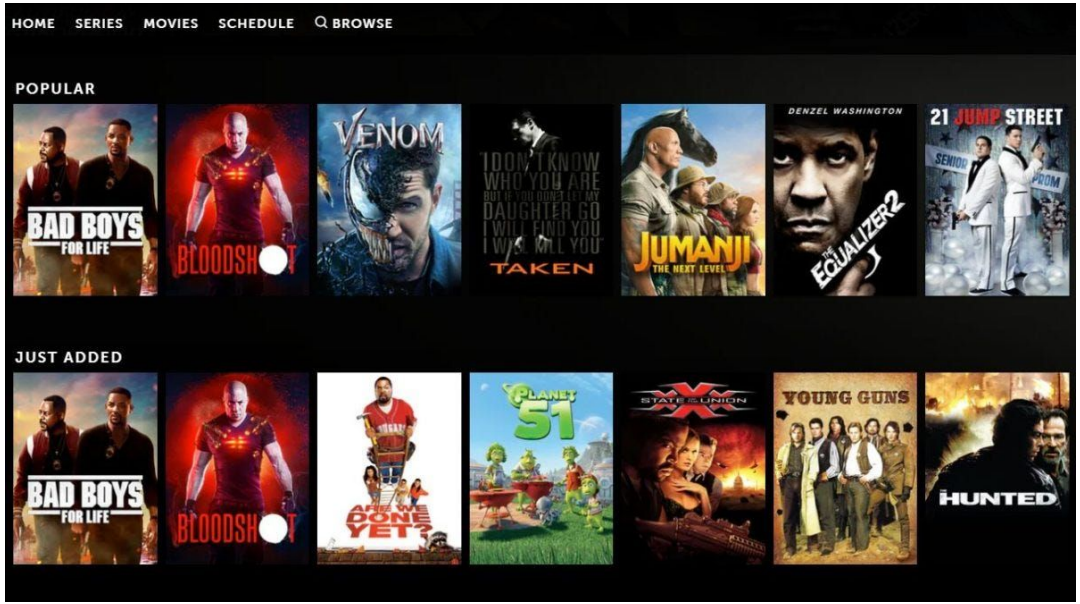


Image credit: Netflix

Successful Recommender System - Instagram

- Instagram friends and posts recommendation
- Instagram suggests **friends**, **pages**, and **posts** based on **user interactions** and **interests**.
- Instagram boosts **user interaction** and **content discovery**.

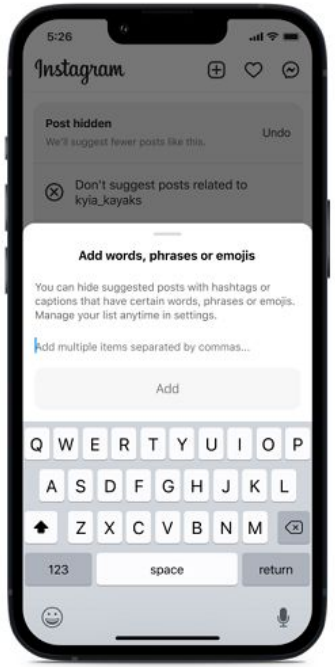


Image credit: Instagram

Successful Recommender System - Google News

- Google News recommendation
- Google News curates articles based on **reading preferences** and **trending topics**.
- Google keeps **users informed** and **engaged with relevant content**.

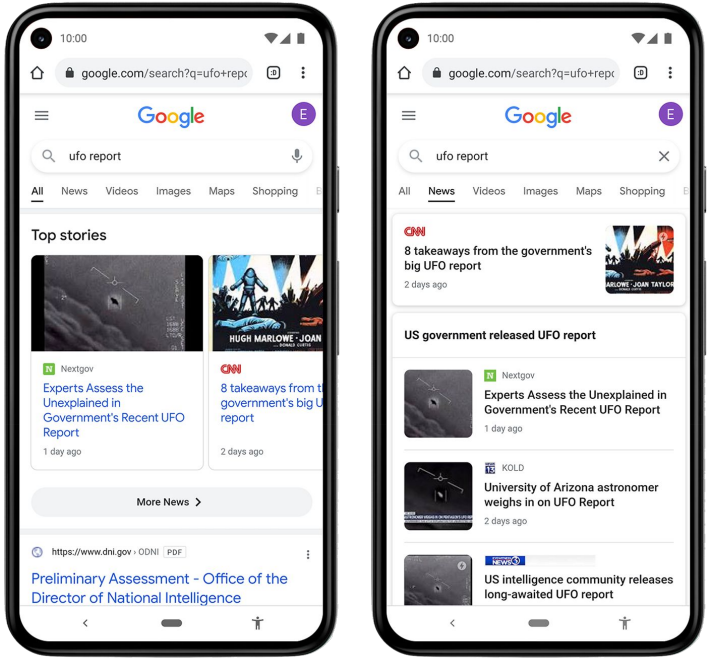


Image credit: Google News

Successful Recommender System - Spotify

- Spotify music recommendation
- Spotify generates **personalized playlists** like “Discover Weekly” based on **listening habits** and **user interactions**.
- Spotify increases **user satisfaction** and **time spent on the platform**.
- Spotify uses a pro-rata model that pools all ad and subscription revenue for the month, **keeps 30%**, and distributes the remaining 70% to artists based on their share of total streams.

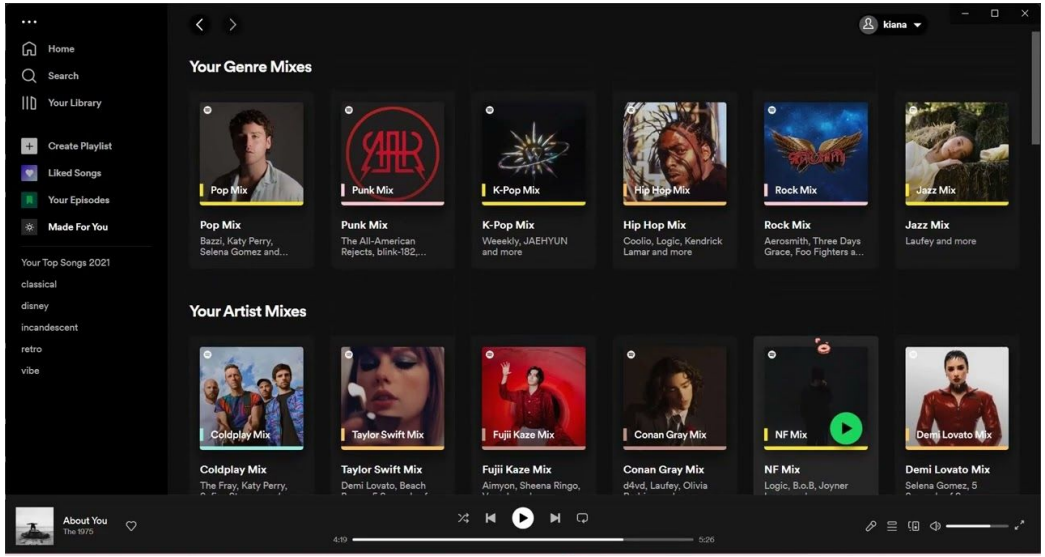


Image credit: Spotify

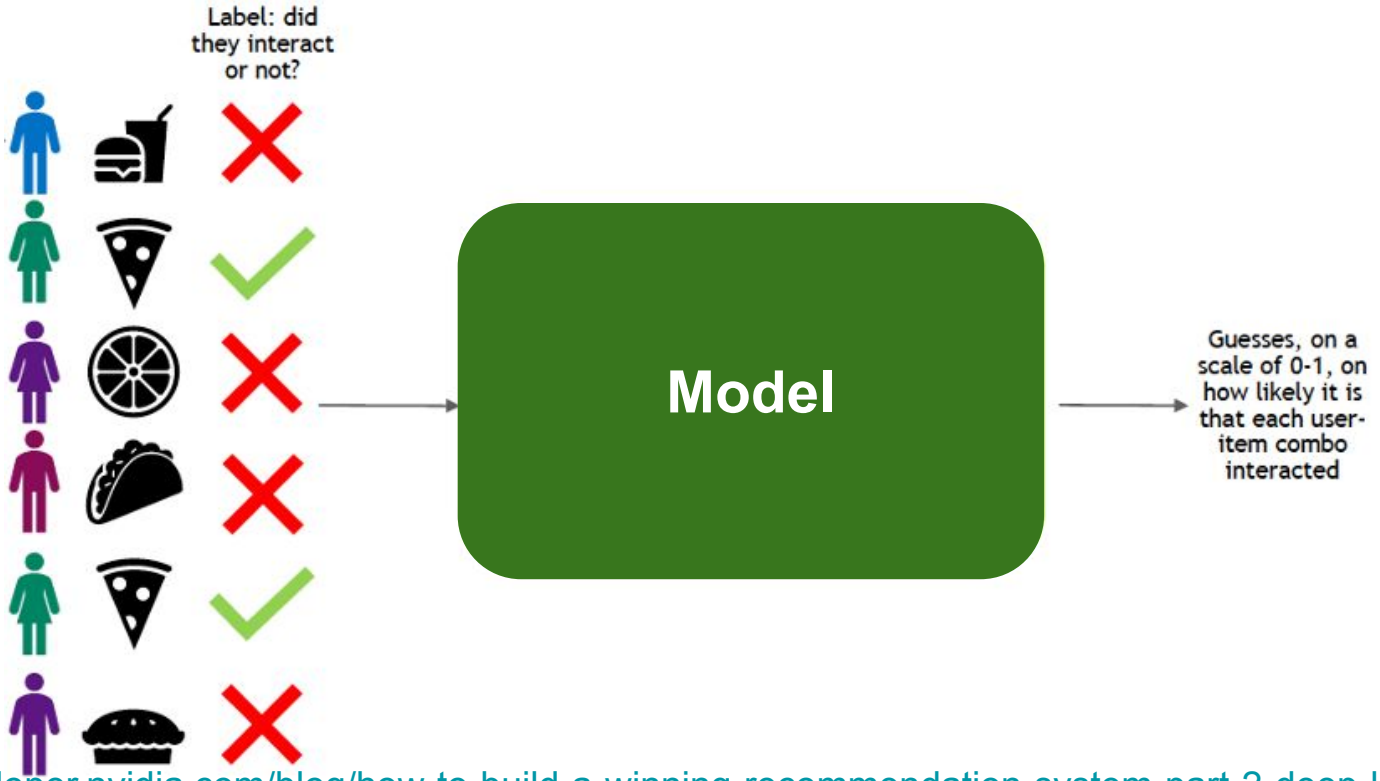
Problem Definition - Recommender System

- A system that uses some decision-making strategies to come up with some suggestions or recommendations

- Importance:
  - Benefits users in finding items of their interest.
  - Help item providers in delivering their items to the right user.
  - Identity products that are most relevant to users.
  - Personalized content.
  - Help websites to improve user engagement.

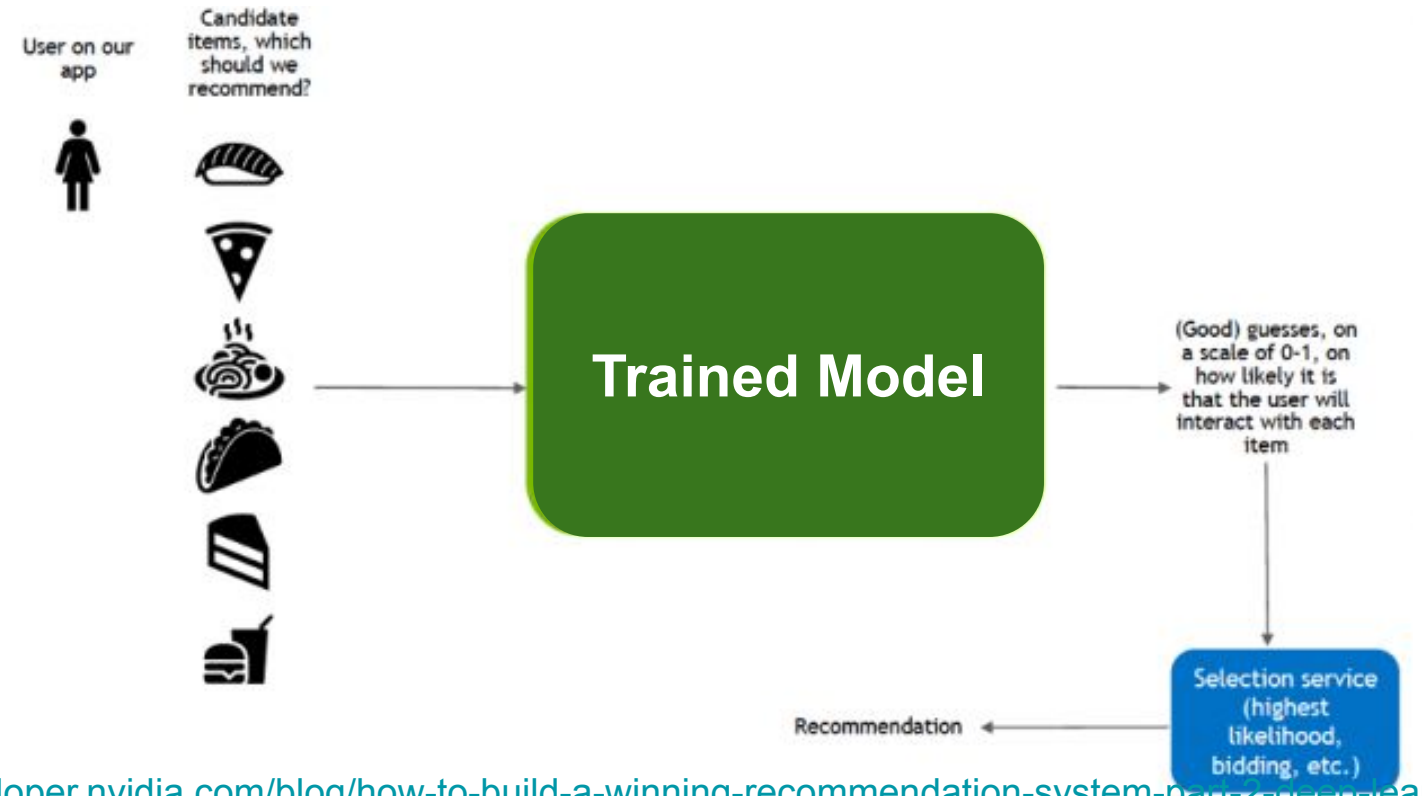
Reference: https://medium.com/@shaonim8/all-about-friend-recommendation-system-cd892e4fa1e0

Training a Recommender System



Reference: https://developer.nvidia.com/blog/how-to-build-a-winning-recommendation-system-part-2-deep-learning-for-recommender-systems/

Using a Recommender System



Reference:
https://developer.nvidia.com/blog/how-to-build-a-winning-recommendation-system-part-2-deep-learning-for-recommender-systems/

Data in Recommender System - e.g. Social Graph

- In graph theory, a graph is defined as (V, E)
- V is a set of vertices, e.g. user
- E is a set of edges, e.g. whether two users are friend or not

Example: Data - `facebook-links.txt`

- Column 1: user A
- Column 2: user B
- Column 3: Timestamp (\N means cannot be determined)
- E.g. first datum: user 1 and user 2 are friends.

1	2	\N
1	3	\N
1	4	\N
1	5	\N
1	6	\N
1	7	\N
1	8	\N
1	9	\N
1	10	\N
1	11	\N
1	12	\N
1	13	\N
1	14	\N
1	15	\N
1	16	\N
1	17	\N
1	18	\N
1	19	\N
1	20	1217964960
1	21	\N
1	22	\N
1	23	\N
1	24	1227241074
1	25	1229314692
1	26	\N
1	27	\N
28	29	\N
28	30	\N
28	31	\N

Use Python NetworkX package to handle graph data

- Official release: https://networkx.org/
- Installation of NetworkX on Anaconda:
  - `conda install -c anaconda networkx --yes`
- Tutorial: https://networkx.org/documentation/networkx-1.9/_downloads/networkx_tutorial.pdf

NetworkX
Network Analysis in Python

Types of Recommender System

1. **First Generation System**: Knowledge-based, Content-based and Collaborative filtering.
2. **Second Generation System**: Hybrid and Matrix Factorization
3. **Third Generation System**: Deep learning based recommender system. For example,
   - Patch-wise Deep Convolutional Neural Network (DeepCoNN)
   - Wide and Deep Network
   - Graph Convolutional Network (GCN)
   - … etc

Code: https://colab.research.google.com/drive/11rJE7dPQi2_JoTHpF5jMT_aemdwQL1Sm

Recommend by Number of Common Friends

- Consider the situation:
  - A has one friend in common with B (namely, C).
  - A has one friend in common with C (namely, B).
  - A has two friends in common with D (namely, B and C).
  - A has no friends in common with E.
  - A has one friend in common with F (namely, G).
- Who will you recommend A as a suggested new friend?
  - The friends of D
  - D has the most number of common friends with A

Recommend by Influence

- What happen, if D has 1 million friends, other users have around 10 friends?
  - A has one friend in common with B (namely, C).
  - A has one friend in common with C (namely, B).
  - A has two friends in common with D (namely, B and C).
  - A has no friends in common with E.
  - A has one friend in common with F (namely, G).
- Would you still recommend A to friends of D?
- Influence score: $\sum_i \frac{1}{\text{numfriend}(f_i)}$, where f_i is the common friends of two users, numfriend(f_i) is the number of friends of f_i.

The Basics for Recommender Systems

- **Collaborative Filtering:** The user-item interactions, such as ratings or buying behaviour, using similarity measure.
  - **User-based**: Recommended movies by someone share similar interests with your

```
user_sim = cosine_similarity(user_item)
sim_users = np.argsort(user_sim[user_id-1])[::-1][1:11]
```

  - **Item-based**: Identify strong connections (e.g. two movies are highly similar), and recommend unwatched movies with similar content

```
item_user = user_item.T
item_sim = cosine_similarity(item_user)
```

The Basics for Recommender Systems

- **Content-based Recommender:** The attribute information about the user and the item such as rating, textual profiles or relevant informations.
  - E.g. Analyze the features of the movie, and recommend movies based on features of watched movies

```
# Step 2: Create TF-IDF Genre Vectors
# Step 3: Get liked movie indices
# Step 4: Average user's liked genres
# Step 5: Cosine similarity
# Step 6: Recommend unrated movies
```

The Basics for Recommender Systems

- **Knowledge-Based Recommender**: The explicit knowledge about user, such as constraints and rules
  - E.g. Recommended by the search keywords, working at the same company, studying in the same programme, .. .etc

```
if 'genres' in preferences:
    pass  # for details, refer to the code

if 'min_year' in preferences:
    pass  # for details, refer to the code
```

Major Limitations for First Generation System

1. **Cold start problem**
   - **New User Problem**: If a new user has no history of ratings or interactions, the system cannot find similar users or make personalized recommendations. The system can only suggest popular or random items.
   - **New Item Problem**: If a new item is added to the catalog, no users have interacted with it yet. Therefore, it can never be recommended by a collaborative filtering system until a sufficient number of users engage with it.
2. **Over-specialization, and hardly recommend new items**: The system recommends items that are extremely similar to the user's current profile. This setup a filter where the user is never exposed to different types of content, especially **recently hot topics**.

Hybrid Recommendation System

- A Hybrid Recommendation System is an approach that combines two or more recommendation techniques to overcome the limitations of individual methods and provide better, more robust recommendations.
  - E.g. Combine collaborative filtering, content-based and knowledge-based

- By combining methods, the limitations of different recommender systems can be addressed.
- However, the ways of combination sounds arbitrary, or requires tuning, or requires trial and error.

Hybrid Recommendation System

1. **Weighted Hybrid**: Combine scores from multiple recommendation algorithms
   - E.g. Final_Score = 0.4 × CF_Score + 0.3 × Content_Score + 0.3 × Knowledge_Score
2. **Switching Hybrid**: Use different methods in different situations
   - E.g. Use content-based for new users, collaborative filtering for established users
3. **Mixed Hybrid**: Present all top recommendations from different algorithms together
4. **Cascade Hybrid**: One technique refines the output of another
   - E.g. Collaborative filtering creates a candidate list. Then, content-based re-ranks the candidates

Matrix Factorization

- Clustering users and items separately does not work well, because

1. We need to **manually inspect** the meaning of each group (cluster)
2. We do not know which user groups correspond to which item groups
3. There is no guarantee that we will obtain exact **correspondence** between user groups and item groups

- E.g. there might be a group of users who like animations, but animations might be grouped under different clusters according to their content

Matrix Factorization

- We can approach this problem differently
- Let's assume in advance that there are K **different types of users and items**
- We use **vectors of length** K to represent users and items
- Each component (**factor**) in a vector represents the **extent** to which the user/item **belongs to that type**

```
user1 = (0.0,0.1,0.2,1.2,2.3)
user2 = (3.1,0.0,0.1,0.2,0.4)

item1 = (0.2,0.9,1.5,0.2,0.8)
item2 = (1.5,2.3,0.0,0.9,0.0)
```

Matrix Factorization

- If we consider all the **users** and **items** in our recommender system, we can represent all of them using two different **matrices**

Users

u_0	1.2	0.1	0.5	0.9	2.3
u_1	0.0	0.2	0.8	2.1	1.7
u_2	.	.	.	.	.
u_3					
u_4					
u_5					

dimension = K

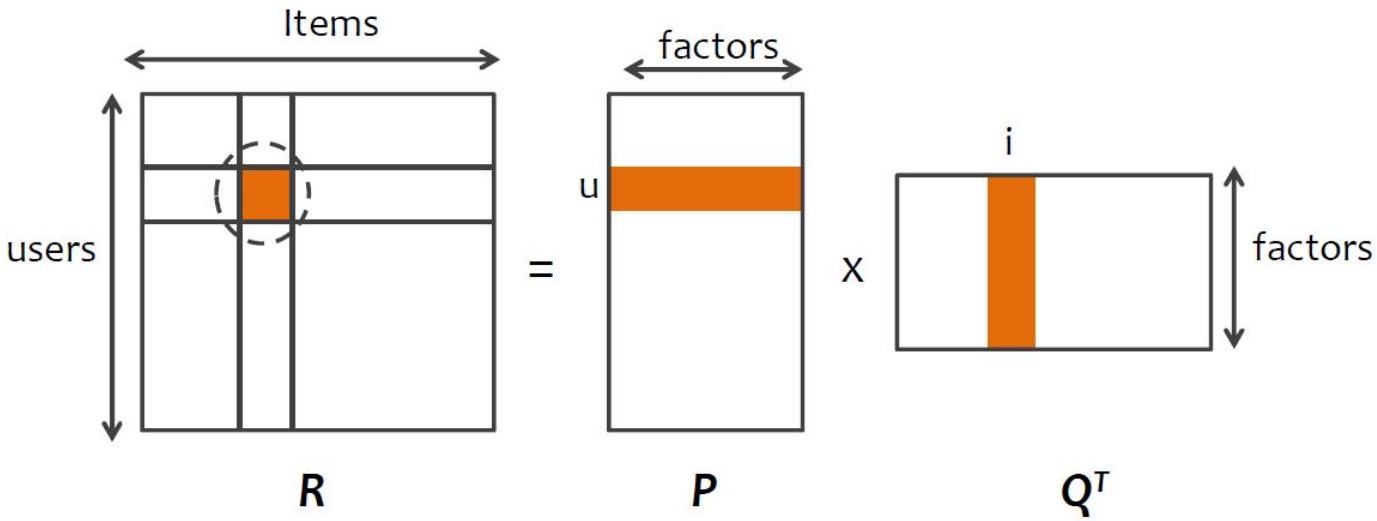
Items

i_0	2.3	0.9	0.2	0.5	0.0
i_1	0.0	0.8	0.7	1.5	1.3
i_2	.	.	.	.	.
i_3					
i_4					
i_5					

dimension = K

Matrix Factorization

- In such a model, the rating on item i given by user u is generated by computing the **matrix product** of the corresponding user and item **vectors**
- This is equivalent to multiplying the (transpose of) the item matrix with the user matrix



Matrix Factorization

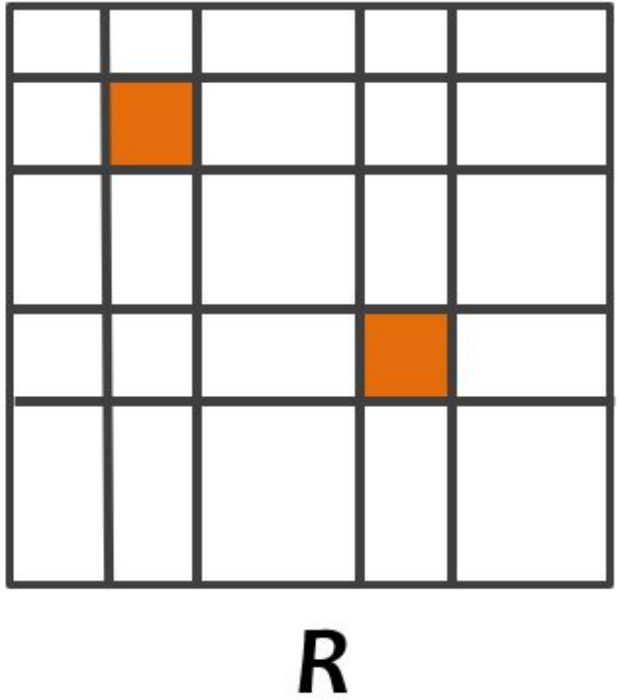
- Usually, we are given the matrix R, **matrix factorization** is the process of finding P and Q such that $P \times Q^T$ gives an approximation of R

$$P \times Q^T = \hat{R} \approx R$$

- Very often R is **incomplete** (e.g. 99% of the cells are empty),
- Therefore, we have to estimate the values in P and Q, instead of solving for exact values
- The number of factors K reflects the complexity of our model

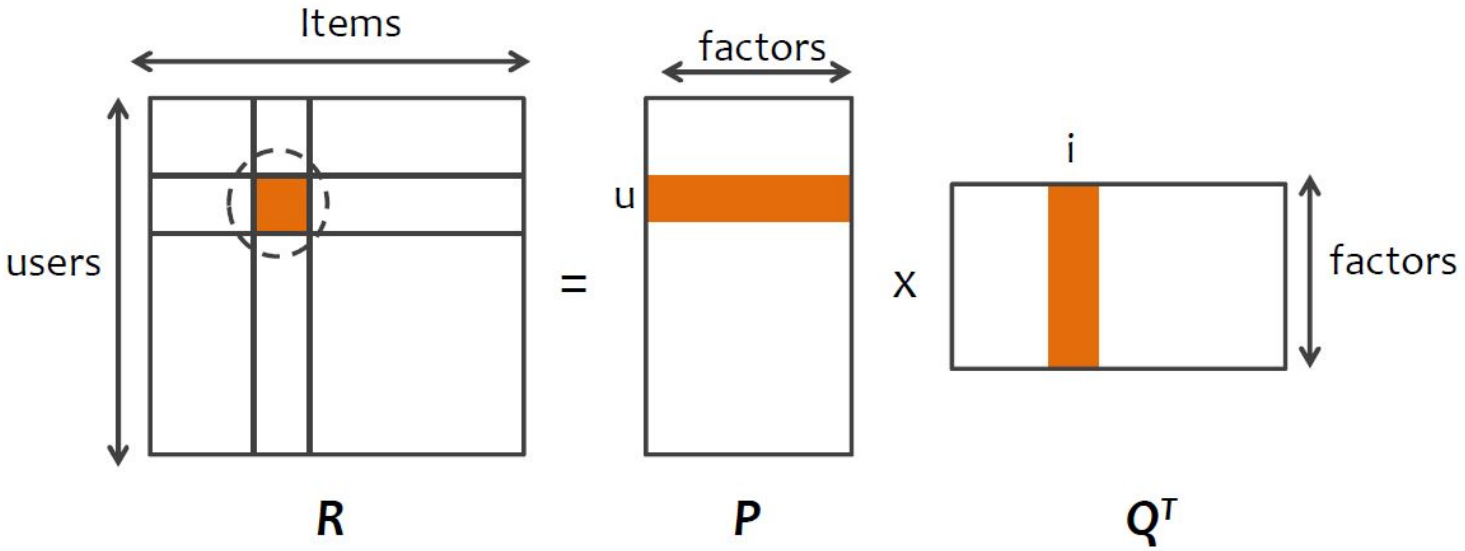
Matrix Factorization

- In matrix factorization, our target is to come up with P and Q, the two **factor matrices** whose product approximates the user-item rating matrix R
- When we say approximate, we are talking about the **known values** in R
- If we can approximate the known values well, our **predictions** of the unknown values will be **accurate**



Matrix Factorization

- Let's focus on **one rating** at a time



$$\hat{r}_{ui} = \sum_{k=1}^{K} p_{uk} q_{ki}$$

Matrix Factorization

- Our goal is to find all p_{ik} and q_{kj} such that

$$\sum_{k=1}^{K} p_{uk} q_{ki} = \hat{r}_{ui} \approx r_{ui}$$

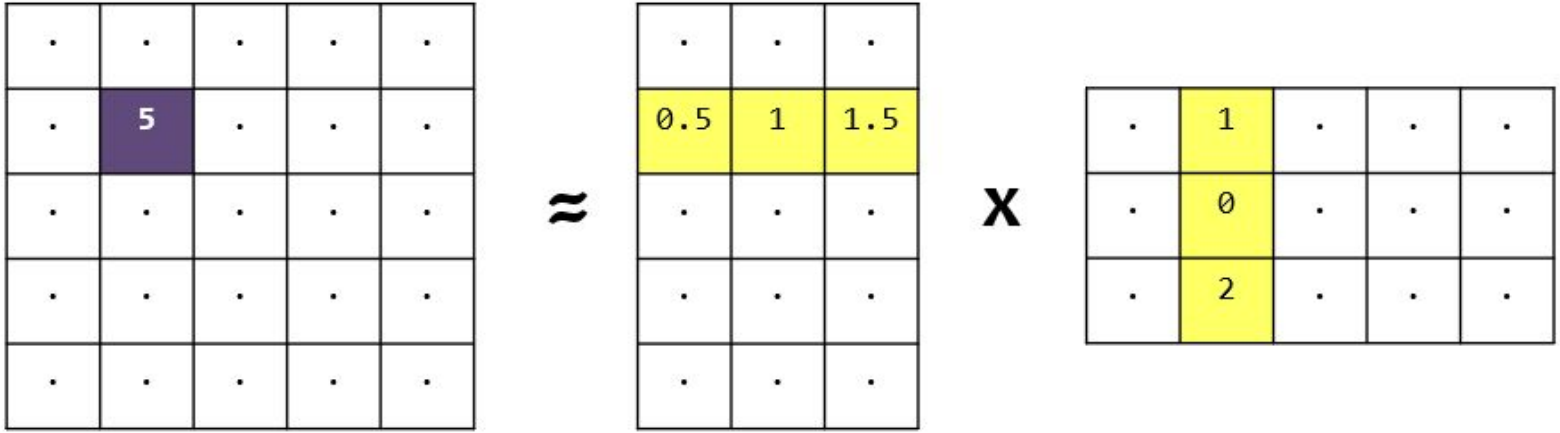
- Our data only contain the values of **some** r_{ij}
- How can we determine the values of all p_{ik} and q_{kj}?

Basic Idea for Matrix Factorization

- We first initialize P and Q with some **random values**
- We measure our **error**, i.e. how far we are from the **true** values in R
- We update the values in P and Q based on the error we computed
- **Repeat** the steps above until the error is sufficiently small

- In other words, we can use **gradient descent**

Matrix Factorization Example



- The **dot product** of the user vector and the item vector is:

  $0.5 \times 1 + 1 \times 0 + 1.5 \times 2 = 3.5$

- We can then compute our **error**

  E.g. absolute error = 5 - 3.5 = 1.5

Error in Matrix Factorization

- Error is give by: $e_{ui} = r_{ui} - \hat{r}_{ui} = r_{ui} - \sum_k p_{uk} q_{ki}$
- Squared error: $e_{ui}^2 = (r_{ui} - \hat{r}_{ui})^2 = (r_{ui} - \sum_k p_{uk} q_{ki})^2$
- The **machine learning** problem here is to optimize all p's and q's such that e_{ui}^2 is **minimized**

Gradient Descent in Matrix Factorization

$$e_{ui}^2 = (r_{ui} - \hat{r}_{ui})^2 = (r_{ui} - \sum_{k=1}^{K} p_{uk} q_{ki})^2$$

$$\frac{\partial e_{ui}^2}{\partial p_{uk}} = -2(r_{ui} - \sum_{k=1}^{K} p_{uk} q_{ki}) q_{ki} = -2 e_{ui} q_{ki}$$

$$\frac{\partial e_{ui}^2}{\partial q_{ki}} = -2(r_{ui} - \sum_{k=1}^{K} p_{uk} q_{ki}) p_{uk} = -2 e_{ui} p_{uk}$$

Gradient Descent in Matrix Factorization

- Having obtained the gradients, we can construct our **update rules** as follows:

$$p'_{uk} = p_{uk} - \alpha(-2e_{ui}q_{ki}) = p_{uk} + 2\alpha e_{ui}q_{ki}$$

$$q'_{ki} = q_{ki} - \alpha(-2e_{ui}p_{uk}) = q_{ki} + 2\alpha e_{ui}p_{uk}$$

- where α is the **learning rate**
- p'_{uk} and q'_{ki} are the **new values**

Training Data in Matrix Factorization

- Note that we only optimize P and Q using the **known values** in the matrix R
- We can consider transforming the user-item rating matrix into a list of **tuples**: (user, item, rating):

.	.	.	3	.
.	5	.	.	2
4	1	.	2	.
.	.	3	.	.
.	.	4	.	5

```
(u0, i3, 3)
(u1, i1, 5)
(u1, i4, 2)
...
...
(u4, i4, 5)
```

Implementation:
1. https://eugeneyan.com/writing/recommender-systems-baseline-pytorch/
2. https://eugeneyan.com/writing/recommender-systems-graph-and-nlp-pytorch/

Example: Basics of Matrix Factorization

```python
def matrix_factorization(R, k, steps=5000, alpha=0.0002, beta=0.02):
    # Initialize P and Q randomly
    P = np.random.normal(scale=1./k, size=(m, k))
    Q = np.random.normal(scale=1./k, size=(n, k))

    # Gradient descent
    for step in range(steps):
        for u in range(m):
            for i in range(n):
                if R[u, i] > 0:  # Only for observed ratings
                    error = R[u, i] - np.dot(P[u, :], Q[i, :])
                    # Update vectors
                    P[u, :] += alpha * (2 * error * Q[i, :] - beta * P[u, :])
                    Q[i, :] += alpha * (2 * error * P[u, :] - beta * Q[i, :])

    return P, Q
```

Example: Basics of Matrix Factorization

- It is more common to use the TruncatedSVD in the sklearn.decomposition package.

```
def train_svd_model(n_components=50):
    svd = TruncatedSVD(n_components=n_components,
        random_state=5731)
    user_factors = svd.fit_transform(user_item)
    item_factors = svd.components_
    return user_factors, item_factors
```

Major Limitations for SVD (Matrix Factorization)

1. **Cold Start**: Still struggles with new users/items (no interaction history)
2. **Linear Assumptions**: Assumes linear combinations of factors
3. **Interpretability**: Latent factors are hard to explain to users, especially we cannot map the latent factors to the physical concepts in the real world

Ideas for using Deep Learning in Recommender Systems

1. Learn the latent variables using deep learning, instead of matrix factorization.
2. Learn the bag-of-words representation using deep learning.
3. Learn the hidden features from user's and item's behaviour using different neural networks (DeepCoNN, Wide and Deep Model, Graph Convolutional Network).

References:

1. https://medium.com/quantyca/deep-learning-for-collaborative-filtering-using-fastai-b28e197ccd59
2. https://proceedings.neurips.cc/paper/2013/file/b3ba8f1bee1238a2f37603d90b58898d-Paper.pdf
3. https://towardsdatascience.com/joint-deep-modeling-of-users-and-items-using-reviews-for-recommendation-50e75d5e10aa

Deep Cooperative Neural Networks (DeepCoNN)

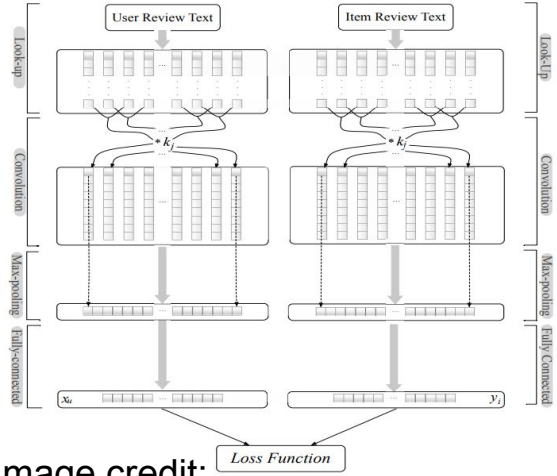


Image credit: https://xdren69.github.io/2021/03/05/reviewRecsys-DeepCoNN/

Deep Cooperative Neural Networks (DeepCoNN)

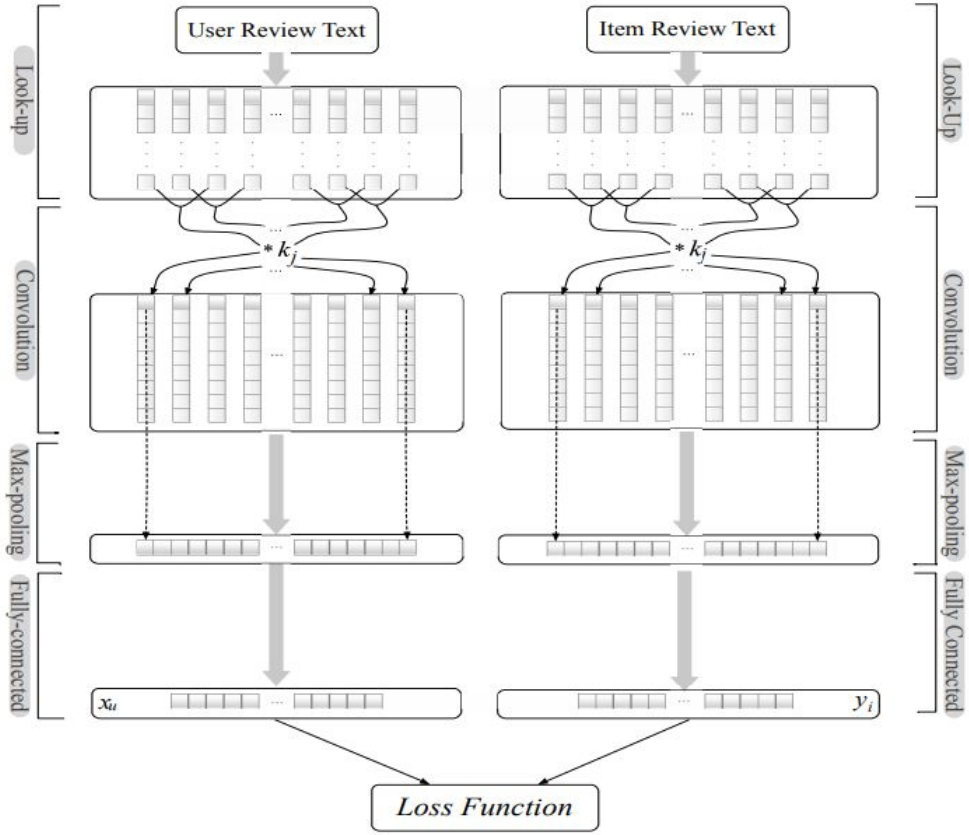


Image credit: https://xdren69.github.io/2021/03/05/reviewRecsys-DeepCoNN/

Deep Cooperative Neural Networks (DeepCoNN)

- Original paper: “Joint Deep Modeling of Users and Items Using Reviews for Recommendation” published in 2017
- DeepCoNN recognizes the textual content of reviews containing information:
  - User preferences and reviewing style
  - Item characteristics and quality aspects
  - The reasons behind ratings
- The major findings include:
  - Directly learned from raw review text
  - Modeled both users and items in a cooperative framework
  - Used parallel neural networks with shared representations

Architecture of DeepCoNN

- DeepCoNN consists of two parallel neural networks that cooperate:

1. **User Network**: Models user behavior from all reviews written by a user
   - User Reviews → Word Embeddings → CNN Layer → Fully Connected → User Latent Features
2. **Item Network**: Models item features from all reviews received by an item
   - Item Reviews → Word Embeddings → CNN Layer → Fully Connected → Item Latent Features
3. **Coupling Layer**: Merge the features to make final ratings or predictions.
   - User Features + Item Features → Fully Connected Layers → Ratings

Example: DeepCoNN - Model Class

```python
class DeepCoNN(nn.Module):
    def __init__(self, n_users, n_movies, embedding_size=50,
        dense_layers=[64, 32, 16]):
        super(DeepCoNN, self).__init__()

        # Embeddings
        self.user_embedding = nn.Embedding(n_users, embedding_size)
        self.movie_embedding = nn.Embedding(n_movies, embedding_size)

        # MLP Layers
        layers = []
        input_size = embedding_size * 2
        for layer_size in dense_layers:
            layers.extend([
                nn.Linear(input_size, layer_size),
                nn.ReLU(),
                nn.Dropout(0.2)
            ])
            input_size = layer_size
        layers.append(nn.Linear(input_size, 1))  # Output
```

Example: DeepCoNN - Model Class

```
        self.mlp = nn.Sequential(*layers)

        # Initialize weights
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, std=0.01)
        elif isinstance(module, nn.Linear):
            nn.init.xavier_normal_(module.weight)
            nn.init.constant_(module.bias, 0)
```

Achievements of DeepCoNN

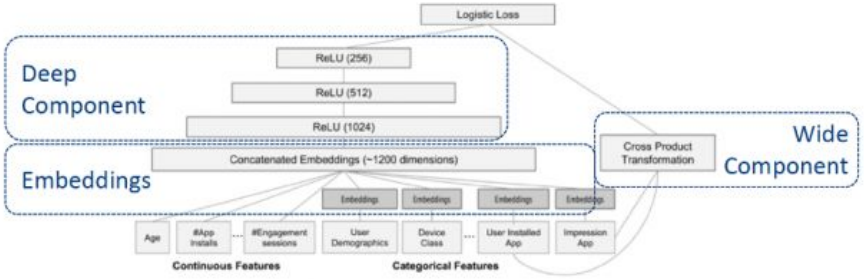
1. DeepCoNN employs deep learning in review-based recommender systems
2. DeepCoNN uses the embeddings of the review texts in constructing the recommender systems, bridging NLP with recommender systems

- The variants of DeepCoNN includes:
  - **TransNets**: Adds a transformation network for better text representation
  - **CARL**: Combines DeepCoNN with contextual information
  - **NARRE**: Neural Attentional Regression with Review-level Explanations

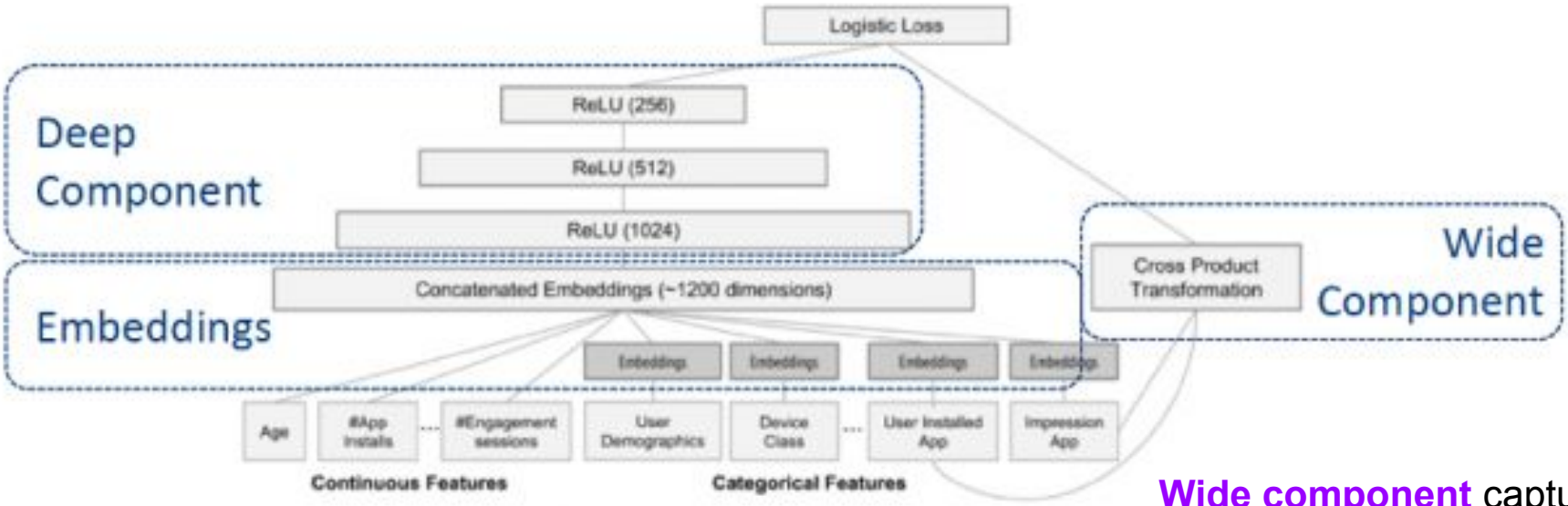
Major Limitations for DeepCoNN

1. DeepCoNN cannot solve the cold start problem.
2. DeepCoNN recommendations are still over-specialized.
3. DeepCoNN is very computationally expensive, compared with the first generation and second generation recommender systems.
4. DeepCoNN inherits the difficulties in NLP.

Wide and Deep Model



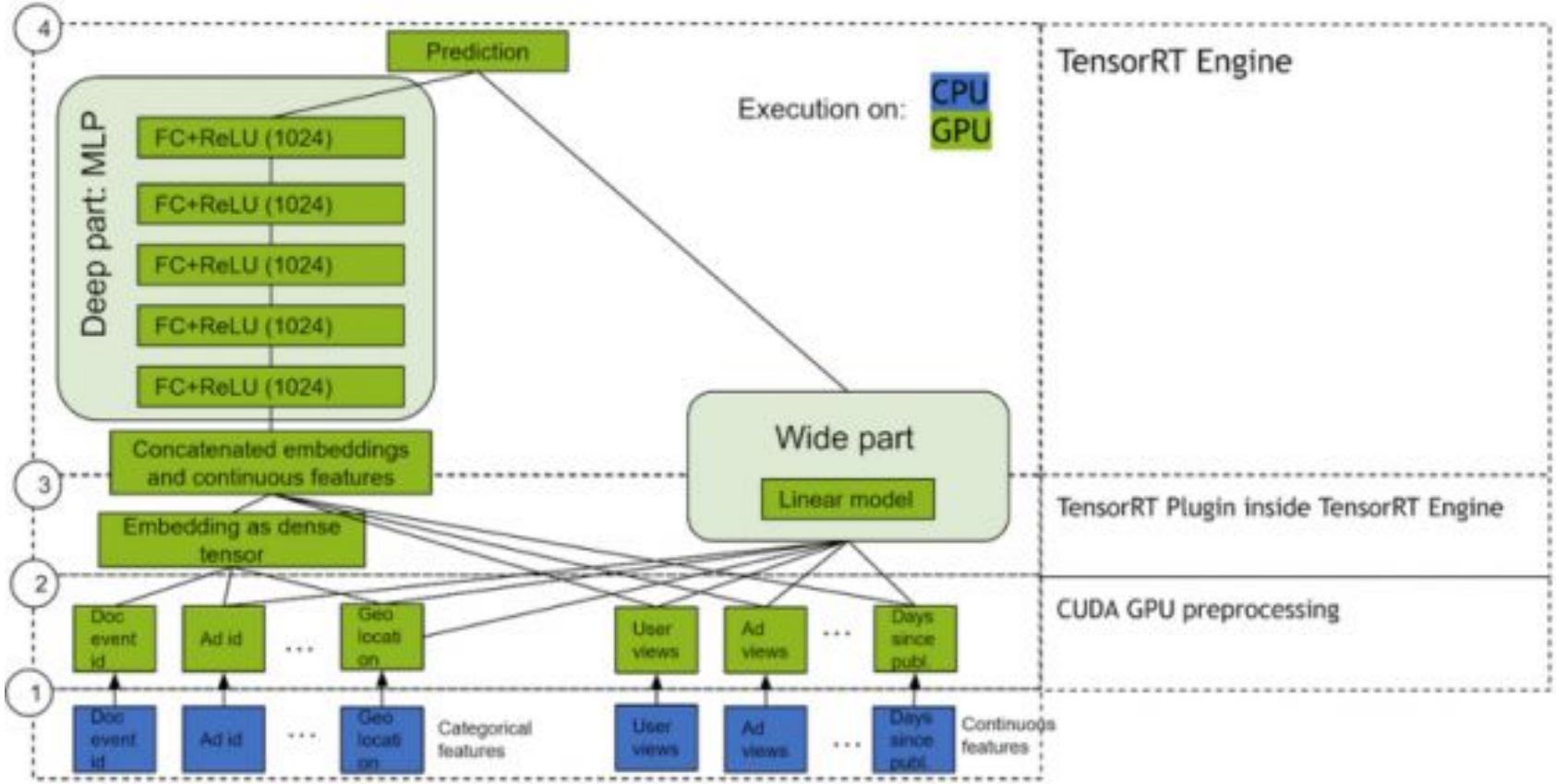
Wide and Deep Model



Deep component can generalize better to unseen feature combinations through low-dimensional dense embeddings learned for the sparse features.

Wide component captures the interactions between the binary features, and adds nonlinearity to the generalized linear model.

Wide and Deep Model



Wide and Deep Model

- Wide and Deep Model is created at Google by Heng-Tze Cheng, Levent Koc, et al. For details, you can refer to the article, “Wide & Deep Learning for Recommender Systems”.
- The primary focus is to improve Google Play Store app recommendation.
- Wide and Deep Model combines the strengths of linear models (memorization) and neural networks (generalization).
- The major contributions include:
  - **Joint Training**: Both paths trained simultaneously, not ensemble
  - **Complementary Strengths**: Wide for memorization, Deep for generalization

Architecture of Wide and Deep Model

1. **Wide Component** - The Memorization Part
   - Learn simple, interpretable patterns and feature interactions
   - Wide Input → Feature Crosses → Linear Model
2. **Deep Component** - The Generalization Part
   - Learn complex patterns and discover new feature combinations
   - Deep Input → Embedding Layers → Hidden Layers → Deep Output
3. **Joint Output**: A single, combined prediction made by fusing the outputs of the Wide and Deep components before the final activation function, for example, weighted sum.
   - Wide Output + Deep Output → Combined Logits → Activation function → Final Prediction

Example: Wide and Deep Model - Model Class

```python
class WideDeep(nn.Module):
    def __init__(self, n_users, n_movies, embedding_size=32, wide_dim=25, deep_layers=[64, 32]):
        super().__init__()
        self.wide = nn.Linear(20, wide_dim)  # 19 genres + 1 year
        self.user_embedding = nn.Embedding(n_users, embedding_size)
        self.movie_embedding = nn.Embedding(n_movies, embedding_size)
        deep_input = embedding_size * 2 + 20
        layers = []
        for layer_size in deep_layers:
            layers.extend([nn.Linear(deep_input, layer_size), nn.ReLU(), nn.Dropout(0.3)])
            deep_input = layer_size
        self.deep = nn.Sequential(*layers)
        self.deep_output = nn.Linear(deep_layers[-1], 1)
        self.final = nn.Linear(wide_dim + 1, 1)
        self.apply(self._init_weights)
```

Achievements of Wide and Deep Model

1. Wide and Deep Model introduced the duality of memorization and generalization.
2. Technical Breakthroughs include:
   - Simultaneous optimization of wide and deep components
   - Single loss function for both components
   - Gradient flow to both paths during backpropagation
3. The foundations of a new model family:
   - Deep & Cross Network (DCN): Automated feature crosses
   - DeepFM: Shared embeddings
   - xDeepFM: Vector-wise interactions
   - AutoInt: Self-attention for features
4. Different proposals are made for the fusion of wide and deep outputs.

Major Limitations for Wide and Deep Model

1. Wide and Deep Model cannot solve the cold start problem.
2. There are scalability issues with the wide component.
3. Wide and Deep Model is very computationally expensive, compared with the first generation and second generation recommender systems.
4. The deep component inherits the difficulties in NLP and multimedia processing.

Graph Convolutional Network (GCN)

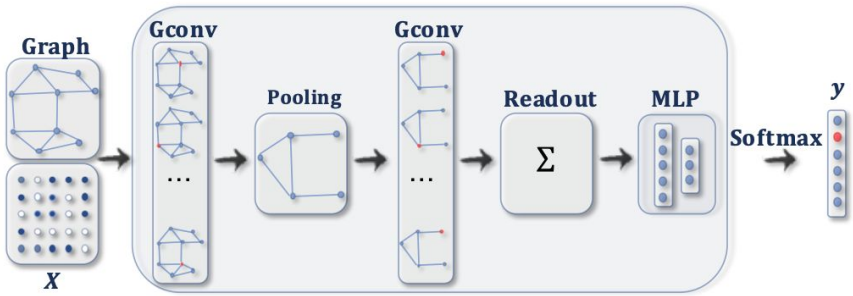


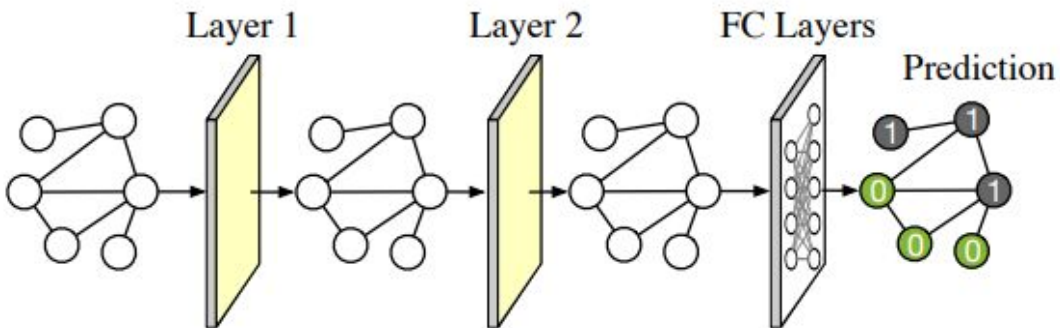
Image credit: Gowri Shankar

Graph Convolutional Network (GCN)

- Graph Convolutional Network (GCN) is originated in the paper, “Semi-Supervised Classification with Graph Convolutional Networks”, published in 2017.
- The major contributions include:
  - Simplified architecture for graph neural network
  - Efficient layer-wise propagation
  - Single model for both node features and graph structure

Graph Convolutional Network (GCN)

- GCN is an approach for semi-supervised learning on graph-structured data.
- GCN is based on an efficient variant of convolutional neural networks which operate directly on graphs.

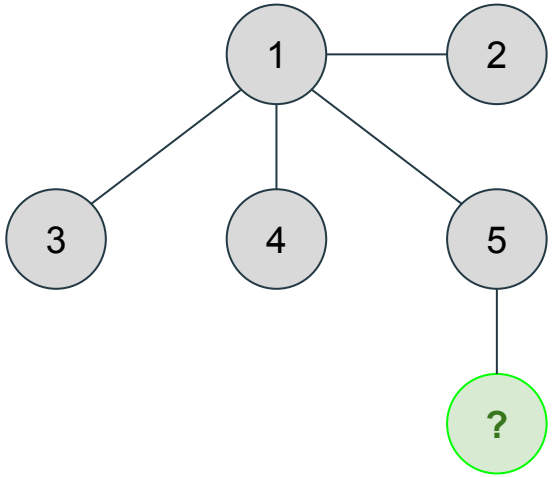


- GCN analyses the graph data in terms of (i) node level, (ii) edge level and/or (iii) graph-level local substructure.

Image credit: https://www.cse.cuhk.edu.hk/~byu/papers/C85-DAC2019-GCN-Test-slides.pdf

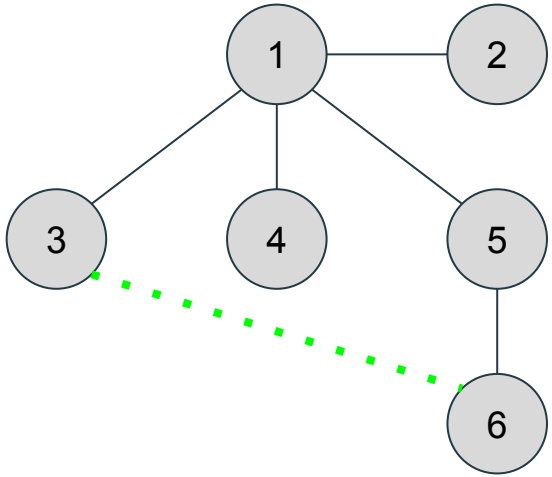
Types of Machine Learning Problems with Graph

Node-level Predictions



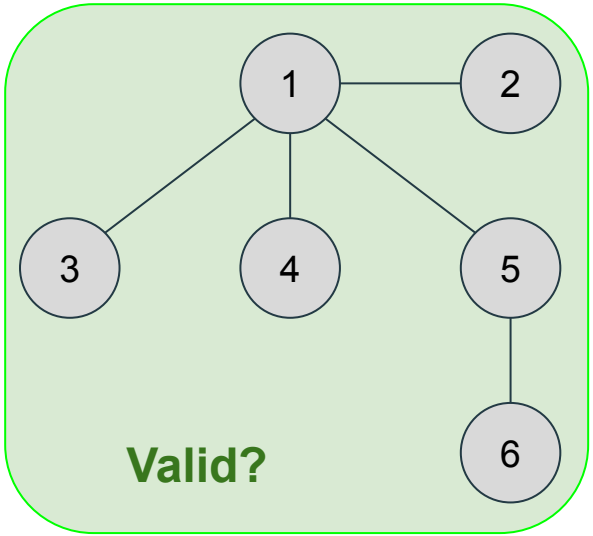
Does this person smoke?

Edge-level Predictions (Link Predictions)



Does person 3 watches movie 6?

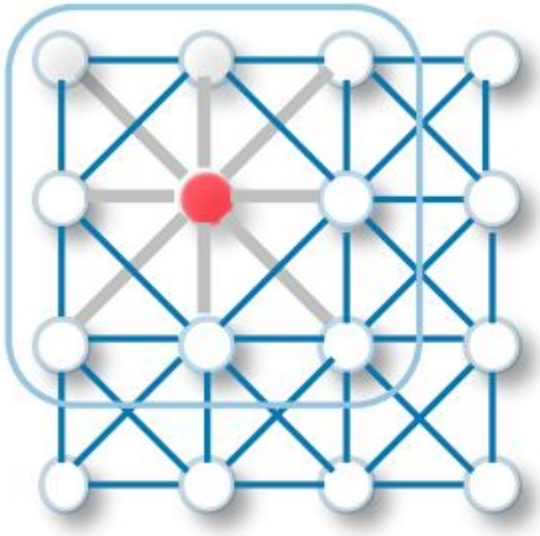
Graph-level Predictions



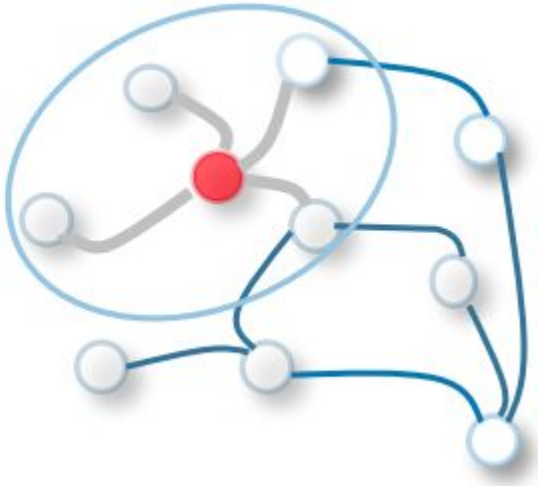
Is this molecule a suitable drug?

Why Convolutional?

Image Convolution



Graph Convolution



Outcomes:

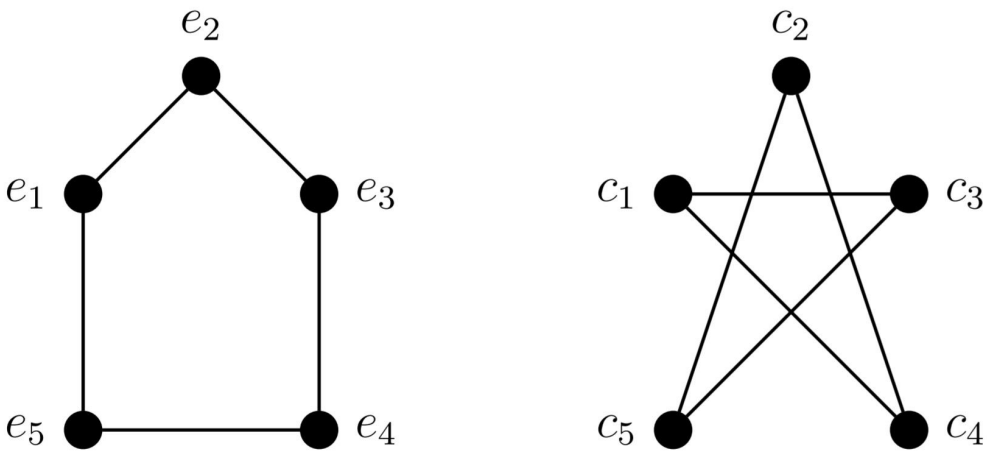
Image pyramid,
Edge or texture

Local structure,
Subnetwork or submodule

Image credit: A comprehensive survey on graph neural network, Wu et al., 2019.

GCN faces more technical difficulties than CNN

1. Graph is hard to resize or reshape.
2. There are isomorphic graphs.
3. Graph is not grid structure, we cannot tell the distance of two nodes, even there is a direct edge connecting them.



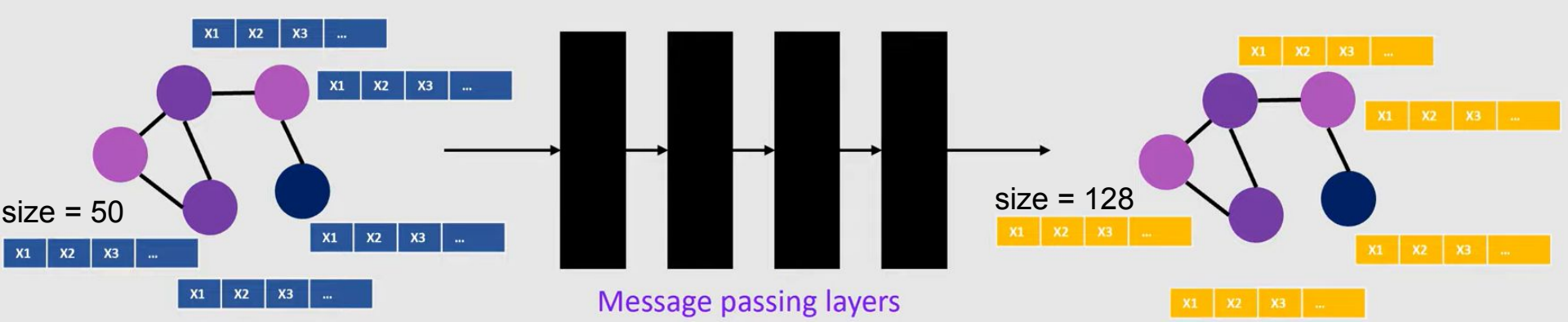
Example:
Isomorphic graph

- $e_1 = c_1$
- $e_2 = c_3$
- $e_3 = c_5$
- $e_4 = c_2$
- $e_5 = c_4$

Image credit: Google Image

The Big Picture of a GCN

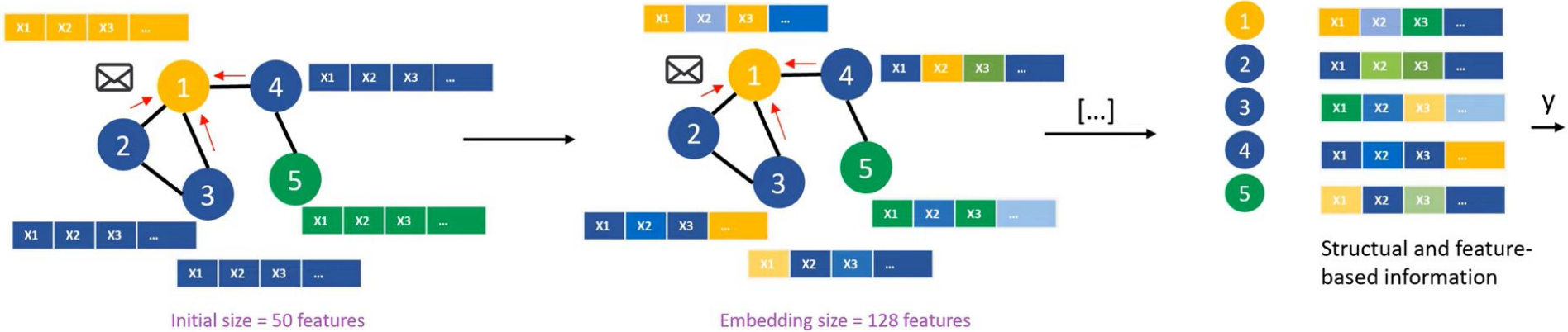
- Learning a forward neural networks suitable representation of graph data.



- We are learning on the **node embeddings** of a graph using GCN.
- In the input, the size (= 50) refers to the properties of the node.
- Throughout training, the size (= 128) refers to the node embeddings.

Image credit:
https://www.youtube.com/watch?v=fOctJB4kVlM&list=RDCMUCScjF2g0_ZNy0Yv3KbsbR7Q&start_radio=1&t=1s&ab_channel=DeepFindr

The Big Picture of a GCN



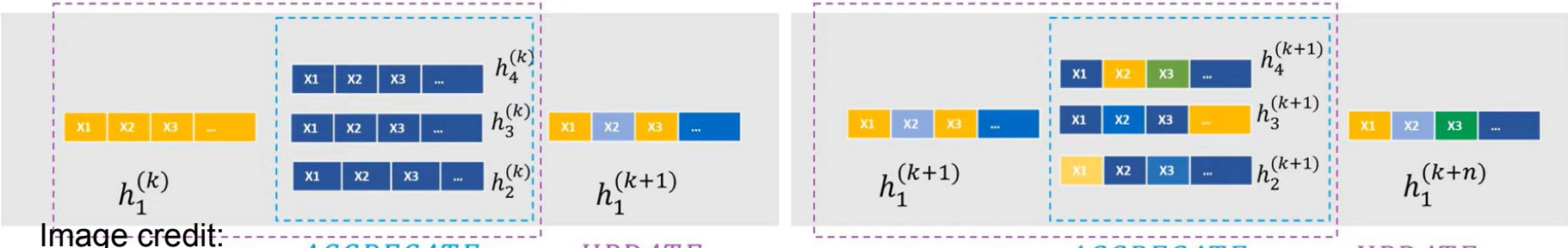
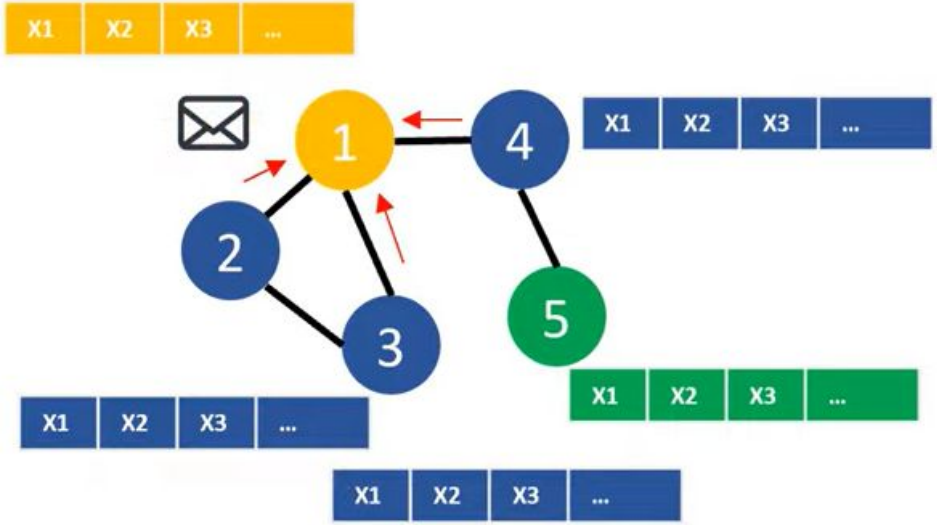
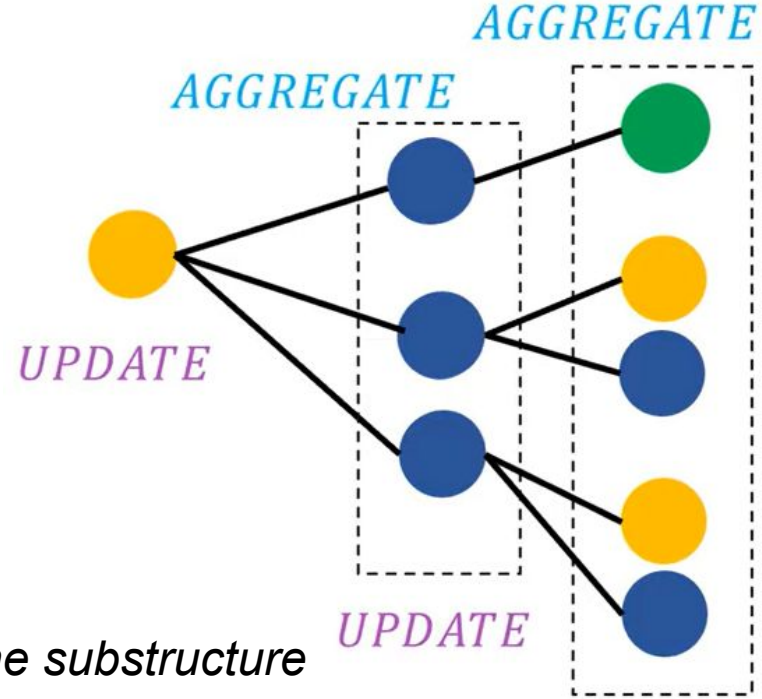


Image credit: https://www.youtube.com/watch?v=ABCGCf8cJOE&list=RDCMUCScjF2g0_ZNy0YV3KbsbR7Q&index=3&ab_channel=DeepFindr

Why do we need multiple layers?





Hyperparameter: Number of layers = size of the substructure

Image credit:
https://www.youtube.com/watch?v=ABCGCf8cJOE&list=RDCMUCScjF2g0_ZNy0Yv3KbsbR7Q&index=3&ab_channel=DeepFindr

Update and Aggregate Functions of GCN

$$h_u^{(k)} = \sigma\left(W_{\text{self}}^{(k)} h_u^{(k-1)} + W_{\text{neigh}}^{(k)} \sum_{v \in N_u} h_v^{(k-1)} + b^{(k)}\right)$$

- $h_u^{(k-1)} \in \mathbb{R}^{d^{(k-1)}}$: Node embeddings
- $W_{\text{self}}^{(k)}, W_{\text{neigh}}^{(k)} \in \mathbb{R}^{d^{(k)} \times d^{(k-1)}}$: Learnable parameters
- $b^{(k)} \in \mathbb{R}^{d^{(k)}}$: Bias term
- σ: Elementwise non-linearity (e.g., a tanh or ReLU)

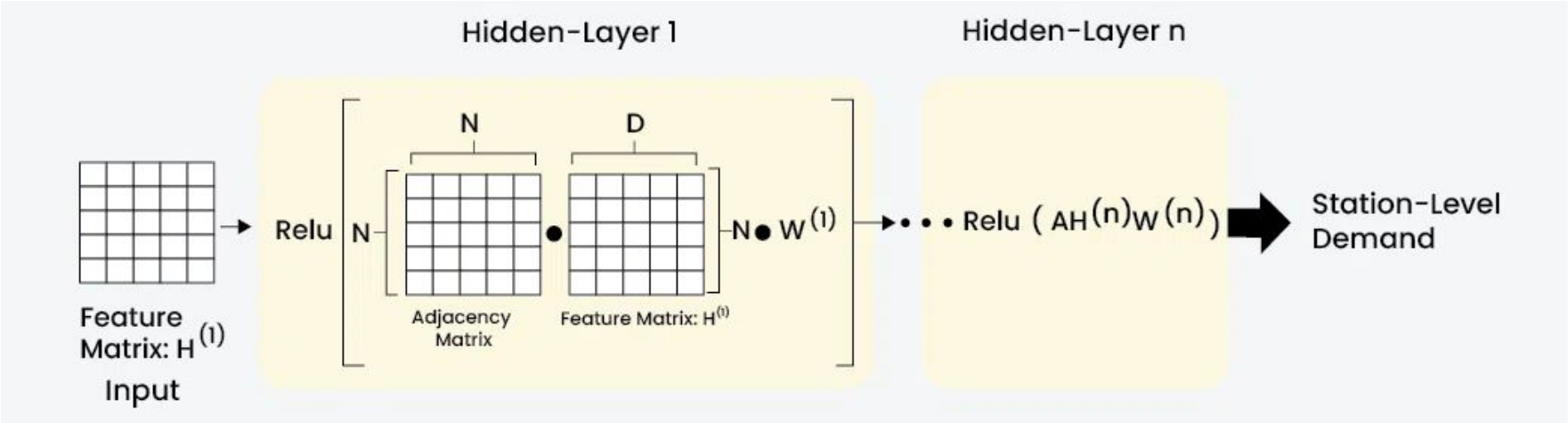
- Note: Both update and aggregate functions are differentiable.
- In this example, update function is σ, while aggregate function is ∑

Architecture for GCN

1. **Input Layer**: The input layer initializes the node features, usually from raw data or pre-trained embeddings.
2. **Hidden Layers**: Hidden layers perform the graph convolution operations, progressively aggregating and transforming node features.
   - **Graph Convolutional Layers**: These layers perform the convolution operation on the graph. Each layer updates the feature representation of a node by aggregating the features of its neighbors.
   - **Activation Functions**: Non-linear functions such as ReLU are applied to the output of each convolutional layer to introduce non-linearity into the model.
   - **Pooling Layers**: These layers reduce the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures.

Architecture for GCN

3. **Output Layer**: The output layer produces the final node embeddings or predictions, depending on the task (e.g., node classification, link prediction).
4. **Fully Connected Layers**: These layers are used at the end of the network to perform tasks such as classification or regression.



Reference:
https://www.geeksforgeeks.org/deep-learning/graph-convolutional-networks-gcns-architectural-insights-and-applications/

Example: A Simple GCN

Reference:
https://github.com/pyg-team/pytorch_geometric

```python
import torch
from torch import Tensor
from torch_geometric.nn import GCNConv
from torch_geometric.datasets import Planetoid

class GCN(torch.nn.Module):
  def __init__(self, in_channels, hidden_channels, out_channels):
    super().__init__()
    self.conv1 = GCNConv(in_channels, hidden_channels)
    self.conv2 = GCNConv(hidden_channels, out_channels)

  def forward(self, x: Tensor, edge_index: Tensor) -> Tensor:
    # x: Node feature matrix of shape [num_nodes, in_channels]
    # edge_index: Graph connectivity matrix of shape [2, num_edges]
    x = self.conv1(x, edge_index).relu()
    x = self.conv2(x, edge_index)
    return x

dataset = Planetoid(root='.', name='Cora')
model = GCN(dataset.num_features, 16, dataset.num_classes)
```

The size of the node embedding is indicated by `hidden_channels`*.*

Another GCN demo:
https://colab.research.google.com/drive/16GBgwYR2ECiXVxA1BoLxYshKczNMeEAQ?usp=sharing#scrollTo=vf_KuYZmnRgn

Example: GCN for Recommendation - Model Class

```python
class GCNRecommender(nn.Module):
    def __init__(self, n_nodes, embedding_dim=64):
        super().__init__()
        self.embedding_dim = embedding_dim
        self.node_embedding = nn.Embedding(n_nodes, embedding_dim)
        self.conv1 = GCNConv(embedding_dim, embedding_dim)
        self.conv2 = GCNConv(embedding_dim, embedding_dim)
        # Structure: 64*2=128 → 128 → 64 → 1
        self.predictor = nn.Sequential(
            nn.Linear(128, 128),  # 64+64=128 input
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )
```

Applications for GCN

1. **Social Network Analysis**: e.g. Facebook Friend Recommendation
2. **Molecular Biology**: e.g. Drug Discovery
3. **Knowledge Graphs**: Example: Entity Classification in Knowledge Graphs
4. **Natural Language Processing (NLP)**: e.g. Semantic Role Labeling
5. **Computer Vision**: e.g Scene Graph Generation, 3D Model Generation

Some Variations on GCN

AGGREGATE
(permutation invariant)

UPDATE

Graph Convolutional Networks, Kipf and Welling [2016]

Self-loop

$$\mathbf{h}_v^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}\right)$$

Sum of normalized neighbor embeddings

Multi-Layer-Perceptron as Aggregator, Zaheer et al. [2017]

Aggregated message

$$\mathbf{m}_{\mathcal{N}(u)} = \boxed{\mathrm{MLP}_\theta}\left(\sum_{v \in N(u)} \mathrm{MLP}_\phi(\mathbf{h}_v)\right)$$

trainable!

Send states through a MLP

Graph Attention Networks, Veličković et al. [2017]

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h}_v$$

Attention weights

$$\alpha_{u,v} = \frac{\exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v]\right)}{\sum_{v' \in \mathcal{N}(u)} \exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}]\right)}$$

Image credit:
https://www.youtube.com/watch?v=ABCGCf8cJOE&list=RDCMUCScjF2g0_ZNy0Yv3KbsbR7Q&index=3&ab_channel=DeepFindr

Major Limitations for GCN

1. GCN cannot solve the cold start problem.
2. GCN encounters scalability issues. It is very difficult to handle a super large social network.
3. The social network (graph) is dynamic. The graph keeps changing.
4. Lots of efforts are working for combining GCN with transformer models, leading to more powerful applications.
5. GCN is very computationally expensive, compared with the first generation and second generation recommender systems.

Non-Traditional Evaluation for Recommender Systems

- **Click-through rate (CTR)**: the number of accepted recommended new friends is divided by the number of friends are recommend
- **Conversion rate**: the percentage of users who have accepted recommended new friends
- **Retention rate**: the percentage of users who continue accepting recommended new friends over a given timeframe
- **Net promoter score (NPS)**: the loyalty of customers to the system ranging from -100 to 100, e.g. subtract the percentage of Detractors from the percentage of Promoters

Stock Market Index Prediction

Image credit: https://corporatefinanceinstitute.com/resources/career-map/sell-side/capital-markets/stock-market-index/

Problem Definition - Stock Market Index Prediction

- To forecast the future stock market trend
  - To raise capital and for individuals to increase wealth
  - To help you know better about your entry and exit points
  - To suggest the investment portfolio

Reference:
https://colab.research.google.com/drive/1d2uaWcNLMgJlgAHY2qp9AnA4PfCO__ce?usp=sharing

Why Stock Market Index Prediction Difficult?

1. **Randomness and "Black Swans"**: High-impact events (a pandemic, a major war, a financial crisis) can shatter the most sophisticated models. In other words, the data are always incomplete and insufficient.
2. **Market Psychology**: In behavioural finance, investors are not always rational, have limits to their self-control and are influenced by their own biases.
3. **Reflexivity**: The act of prediction can influence the market itself. If a famous analyst predicts a crash, it might cause a sell-off, making the prediction self-fulfilling.
4. **Overfitting**: A major problem in ML models. A model can be trained to perfectly predict past data but fail miserably with future data because it learned the noise instead of the underlying signal.

Data Source - Stock Market Index Prediction

- To obtain the yahoo finance website data via Python `yfinance` API
- This API gathers all of the historical stock market data

```python
# 1: Download historical stock data
ticker = yf.Ticker("AAPL")          # Apple Inc.
data = ticker.history(period="1y")  # 1 year of data
print("Historical Data:")
print(data.head())
# Download with different periods
data_daily = ticker.history(period="6mo")                    # 6 months
data_weekly = ticker.history(period="6mo", interval="1wk")  # Weekly
data_monthly = ticker.history(period="2y", interval="1mo")  # Monthly
```

```
Historical Data:
                                  Open        High         Low       Close  \
Date
2024-10-18 00:00:00-04:00  235.085888  235.085888  232.925942  233.911362
2024-10-21 00:00:00-04:00  233.363896  235.752787  233.363896  235.384491
2024-10-22 00:00:00-04:00  232.806520  235.125728  231.522502  234.767395
2024-10-23 00:00:00-04:00  232.995622  234.050709  226.704892  229.690994
2024-10-24 00:00:00-04:00  228.914614  229.750734  227.351895  229.501892
```

Data Source - Stock Market Index Prediction

```python
# 2. MULTIPLE TICKERS
print("\n=== 2. MULTIPLE TICKERS ===")
tickers = ["AAPL", "MSFT", "GOOGL", "TSLA"]
multi_data = yf.download(tickers, start="2024-01-01", end="2024-10-18")
print(f"Multi-ticker shape: {multi_data.shape}")
print(multi_data.head())
```

```
=== 2. MULTIPLE TICKERS ===
[*********************100%***********************]  4 of 4 completed
Multi-ticker shape: (201, 20)
Price             Close                                          High  \
Ticker             AAPL       GOOGL        MSFT        TSLA        AAPL
Date
2024-01-02  184.081497  137.223572  366.105560  248.419998  186.857993
2024-01-03  182.703156  137.968414  365.839050  238.449997  184.319476
2024-01-04  180.382843  135.455750  363.213165  237.929993  181.552929
2024-01-05  179.658936  134.800247  363.025604  237.490005  181.225673
2024-01-08  184.002167  137.888962  369.876495  240.449997  184.041839
```

```
Price                                                   Low                \
Ticker           GOOGL        MSFT        TSLA        AAPL       GOOGL
Date
2024-01-02  138.494803  371.070940  251.250000  182.346189  135.545145
2024-01-03  138.673558  368.464882  245.679993  181.890032  136.141022
2024-01-04  138.206780  368.306880  242.699997  179.361491  135.416030
2024-01-05  136.220460  367.280232  240.119995  178.657420  134.224218
2024-01-08  138.057795  370.379953  241.250000  179.976254  135.326633
Price                                     Open                           \
Ticker            MSFT        TSLA        AAPL       GOOGL        MSFT
Date
2024-01-02  362.058226  244.410004  185.578815  137.600974  369.057139
2024-01-03  363.775903  236.320007  182.673409  136.309855  364.269480
2024-01-04  362.453068  237.729996  180.620818  137.471843  365.908104
2024-01-05  361.791663  234.899994  180.462148  135.813264  364.229932
2024-01-08  364.269471  235.300003  180.561297  135.356426  364.555724
Price                      Volume
Ticker            TSLA      AAPL     GOOGL      MSFT       TSLA
Date
2024-01-02  250.080002  82488700  23711200  25258600  104654200
2024-01-03  244.979996  58414500  24212100  23083500  121082600
2024-01-04  239.250000  71983600  27137700  20901500  102629300
2024-01-05  236.860001  62379700  22513900  21004600   92488900
2024-01-08  236.139999  59144500  21404000  23134000   85166600
```

Data Source - Stock Market Index Prediction

Field	Meaning	Example, on Oct 18, …
Open	First traded price of the period	9:30 AM: Market opens, first trade = $233.07
High	Highest price reached during period	During day: Price peaked at $233.99
Low	Lowest price reached during period	During day: Price dropped to $230.17
Close	Last traded price of the period	4:00 PM: Market closes, last trade = $230.64

Date	Open	High	Low	Close
2024-10-18	233.07	233.990005	230.169998	230.640015
2024-10-17	233.33	235.149994	231.759995	233.000000
2024-10-16	228.88	230.280029	227.070007	228.870010

Data Source - Stock Market Index Prediction

```python
# 3. COMPANY INFO
print("\n=== 3. COMPANY INFORMATION ===")
info = ticker.info
print(f"Company: {info['longName']}")
print(f"Sector: {info['sector']}")
print(f"Market Cap: ${info['marketCap']:,.0f}")
print(f"52W High: ${info['fiftyTwoWeekHigh']:.2f}")
```

```
=== 3. COMPANY INFORMATION ===
Company: Apple Inc.
Sector: Technology
Market Cap: $3,744,081,903,616
52W High: $260.10
```

Data Source - Stock Market Index Prediction

```python
# 3. DIVIDENDS & SPLITS
print("\n=== 4. DIVIDENDS & SPLITS ===")
dividends = ticker.dividends
splits = ticker.splits
print("Recent Dividends:")
print(dividends.tail())
print("\nSplits:")
print(splits.tail())
```

```
=== 4. DIVIDENDS & SPLITS ===
Recent Dividends:
Date
2024-08-12 00:00:00-04:00    0.25
2024-11-08 00:00:00-05:00    0.25
2025-02-10 00:00:00-05:00    0.25
2025-05-12 00:00:00-04:00    0.26
2025-08-11 00:00:00-04:00    0.26
Name: Dividends, dtype: float64
```

```
Splits:
Date
1987-06-16 00:00:00-04:00    2.0
2000-06-21 00:00:00-04:00    2.0
2005-02-28 00:00:00-05:00    2.0
2014-06-09 00:00:00-04:00    7.0
2020-08-31 00:00:00-04:00    4.0
Name: Stock Splits, dtype: float64
```

Data Source - Stock Market Index Prediction

```python
# 5. FINANCIAL STATEMENTS
print("\n=== 5. FINANCIAL STATEMENTS ===")
income = ticker.financials
balance = ticker.balance_sheet
cashflow = ticker.cashflow
print("Income Statement (Latest Year):")
print(income.iloc[:, 0].head())
```

```
=== 5. FINANCIAL STATEMENTS ===
Income Statement (Latest Year):
Tax Effect Of Unusual Items                                    0.000000e+00
Tax Rate For Calcs                                             2.410000e-01
Normalized EBITDA                                              1.346610e+11
Net Income From Continuing Operation Net Minority Interest     9.373600e+10
Reconciled Depreciation                                        1.144500e+10
Name: 2024-09-30 00:00:00, dtype: float64
```

Data Source - Stock Market Index Prediction

```python
# 5. ANALYST RECOMMENDATIONS
print("\n=== 6. ANALYST DATA ===")
recommendations = ticker.recommendations
print(recommendations.tail())
```

```
=== 6. ANALYST DATA ===
  period  strongBuy  buy  hold  sell  strongSell
0     0m          5   23    15     2           3
1    -1m          5   23    15     1           3
2    -2m          5   22    15     1           1
3    -3m          5   23    15     1           1
```

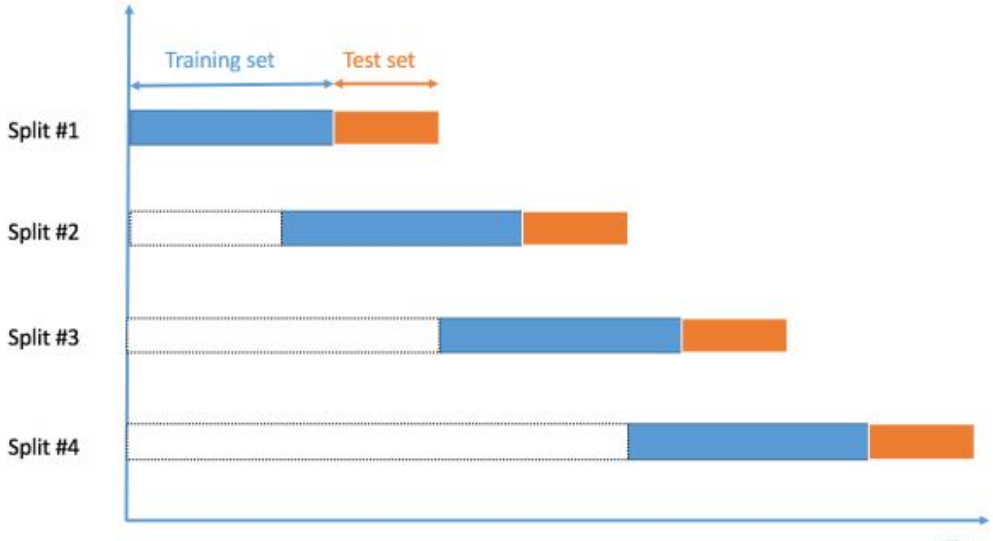
Common Patterns in yfinance

```
ticker = yf.Ticker("AAPL")

ticker.history(period="1y")                    # Daily OHLCV
ticker.info                                    # Company fundamentals
ticker.dividends                               # Dividend history
ticker.financials                              # Income statement
ticker.balance_sheet                           # Balance sheet
yf.download("AAPL MSFT", period="6mo")         # Multiple tickers
ticker.history(period="5d", interval="1m")     # Intraday data
```

Data Split - Stock Market Index Prediction

1. **The Simple Train-Test Split** (The Holdout Method): Reserve a portion of your data at the end of the series for testing.
2. **Rolling Window / Time Series Cross-Validation**:
   - The training set is a **fixed size**, and it "slides" forward in time, dropping the oldest data points as it adds new ones.
   - Alternatively, the training set starts small and **grows over time**, always beginning from the first date. The test set is a fixed-size window that moves forward.



Simplified Stock Market Index Prediction

1. Data exploration (all data are numerical)
2. Data split (cross validation on time series data)
3. Model training
    - Random Forest Model
    - Binary classification, either up or down
4. Model assessed using accuracy
5. Interpreting the results with plots

Model Performance

```
Data shape: (2466, 5)
Ready data shape: (2446, 16)

Features created:
- Range, Body, Gap (OHLC derived)
- SMA_5, SMA_20, RSI, Returns
- Close/SMA ratios, Volatility
- Target: 1=UP, 0=DOWN

MODEL ACCURACY: 52.7%

CLASSIFICATION REPORT:
              precision    recall  f1-score   support
           0       0.47      0.40      0.43       221
           1       0.56      0.63      0.59       269
    accuracy                           0.53       490
   macro avg       0.52      0.52      0.51       490
weighted avg       0.52      0.53      0.52       490
```

```
TOP FEATURES:
       Feature  Importance
8  Close_SMA20    0.107163
6      Returns    0.102975
0        Range    0.102522
9   Volatility    0.102435
2          Gap    0.102354

TOMORROW PREDICTION:
Direction: UP
Probability UP: 58.0%
```

Code: https://colab.research.google.com/drive/1d2uaWcNLMgJlgAHY2qp9AnA4PfCO__ce#scrollTo=fEHRNxcX--I7

Discussions for the Simple Approach

- Do we care the accuracy of the model in this prediction task?
- Is this a binary classification problem indeed?
- How can we earn money after training these models?
- Is the data exploration enough?
- Is the data coverage enough?
- There are some other ANN models, Deep-NN models nowadays.
- Of course, if it really makes money, the work will not be published.

Classic Stock Market Index Prediction

- Some common time-series forecasting models:

1. Autoregressive (AR) model
2. Moving average (MA) model
3. Autoregressive moving average (ARMA) model
4. Autoregressive integrated moving average (ARIMA) model
5. Seasonal autoregressive integrated moving average (SARIMA) model
6. Vector autoregressive (VAR) model
7. Vector error correction (VECM) mode

Moving Average (MA)

- MA is a simple way to smooth out price data or any other data series to identify the underlying trend.

1. **Simple Moving Average (SMA)**: SMA = (Sum of Prices for last n periods) / n
2. **Exponential Moving Average (EMA)**: EMA gives more weight to recent prices and less weight to older prices, reacting faster to the recent information.

Classic Stock Market Index Prediction

1. Data exploration (all data are numerical)
2. Data split using the Holdout Method
3. Model training
    - Exponential Moving Average Model
    - Regression problem
4. Model assessed using MAE and R^2
5. Interpreting the results with plots

Model Performance

```
Downloading SPY data...
Total data: 2466 days
Train: 2014 days (2015-2022)
Test:  452 days (2023-2024)

🔧 Creating regression features...
Features: ['EMA_Diff', 'EMA_Ratio', 'Price_EMA12', 'Moment
Target: Tomorrow's % return

Training Linear Regression...

TRAINING RESULTS:
   MAE: 0.0075 % (avg error)
   R²:  0.006 (explained variance)

TESTING RESULTS:
   MAE: 0.0061 % (avg error)
   R²:  -0.008 (explained variance)
```

```
BACKTEST RESULTS (2023-2024):
Buy & Hold:    +45.3%
EMA Regression:+-11.9%
Edge:          +-57.2%
```

Code: https://colab.research.google.com/drive/1d2uaWcNLMgJlgAHY2qp9AnA4PfCO__ce#scrollTo=FHuapawhfo-S

Discussions for the Classic Approach

- Indeed, moving average is a lagging indicator.
- The EMA assumes the trend will continue, which is not always true.
- Is the data coverage enough? The EMA cannot account for:
  - Company fundamentals (earnings, revenue, debt).
  - Macroeconomic data (GDP, inflation, employment).
  - Market sentiment (news, social media, investor fear/greed).
  - “Black Swan” events (pandemics, wars, financial crises).
- There are some other ANN models, Deep-NN models nowadays.
- Once again, if it really makes money, the work will not be published.

Long Short-Term Memory (LSTM) Model

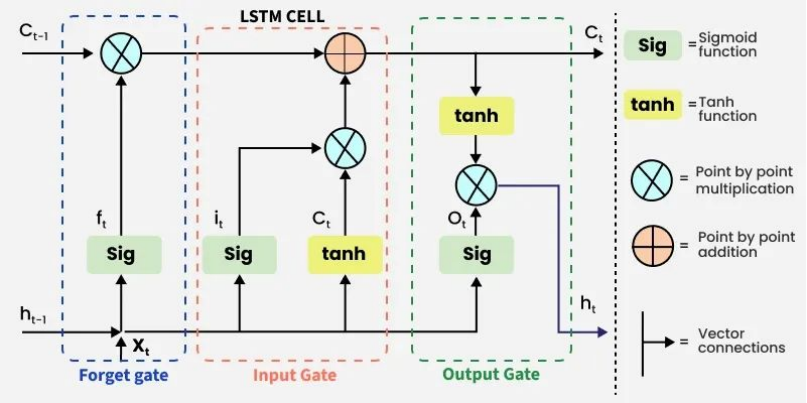


Image credit: Geeks For Geeks

Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) Models

- In 1982, John Hopfield popularized a type of recurrent network, known as Hopfield Network.
- In 1986, Michael I. Jordan introduced the Jordan Network, a simplified RNN architecture. At the same time, the backpropagation algorithm is independently rediscovered by David Rumelhart.
- In 1991, the vanishing gradient problem in the backpropagation through time is analysed and demonstrated by Sepp Hochreiter.
- In 1997, Long Short-Term Memory is first invented by Jürgen Schmidhuber.
- Due to the advancement of the computation power by GPUs in 2010s, both RNN and LSTM get popular for text generation, image captioning and video analysis.

Recurrent Neural Network (RNN)

- A class of artificial neural networks where connections between nodes form a directed or undirected graph along a temporal sequence, to exhibit temporal dynamic behavior.
- Derived from feedforward neural networks
- Applicable to tasks such as unsegmented, connected handwriting recognition or speech recognition.

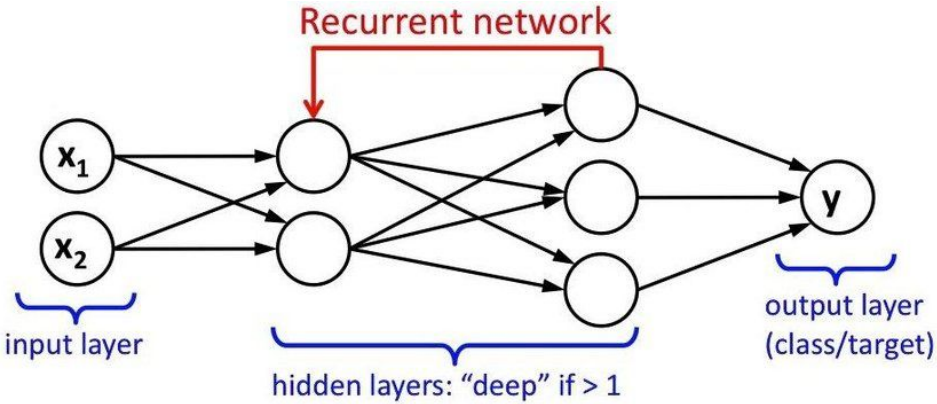


Image credit:
https://www.researchgate.net/figure/Recurrent-neural-networkRNN-or-Long-Short-Term-MemoryLSTM-5616_fig2_324883736

More about Recurrent Neural Network

- We can process a sequence of vectors x by applying a recurrence formula at every time step:

  $$h_t = f_W(h_{t-1}, x_t)$$

- h_t: new state
- h_{t-1}: old state
- f_W: some activation functions with parameters W
- x_t: input vector at some time point

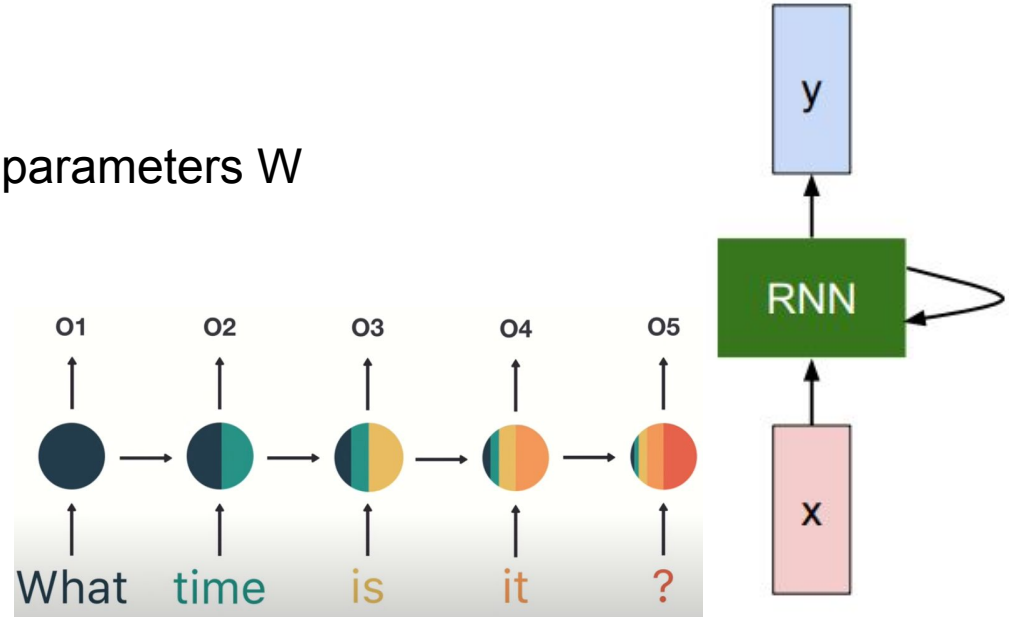


Image credit: https://www.youtube.com/watch?v=LHXXI4-IEns&ab_channel=TheA.I.Hacker-MichaelPhi

More about Recurrent Neural Network

- RNN is capable of handling stateless information.
  - You can apply fixed-width sliding window on RNN, so that:
    - The size of the input = the size of the output
- The self-attention mechanism in RNN promotes the connection of the knowledge from different sliding windows.

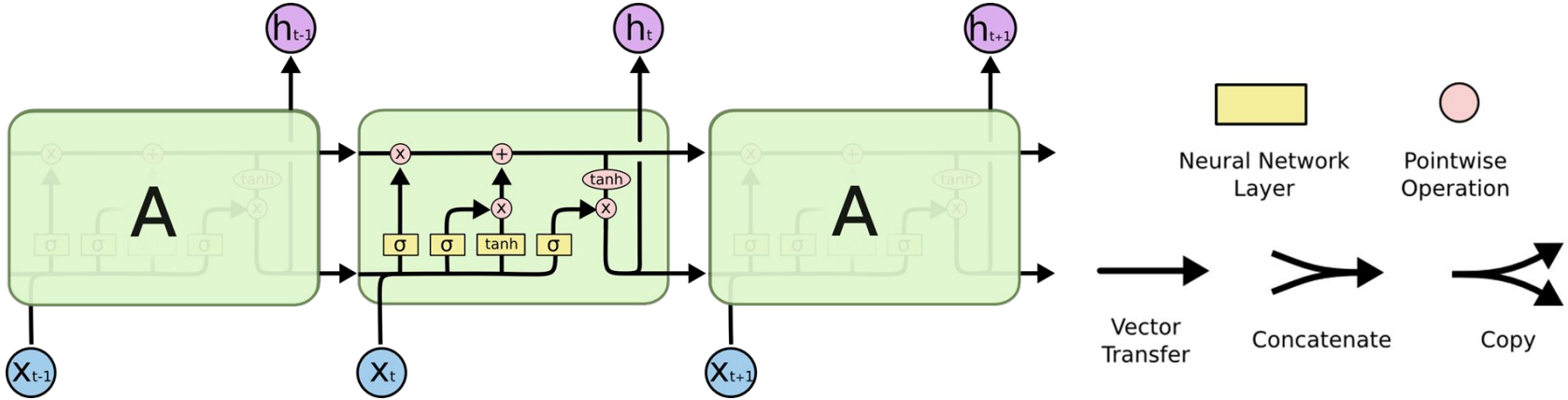
Example: A Simple PyTorch RNN

```python
import torch
import torch.nn as nn
rnn = nn.RNN(10, 20, 2)
input = torch.randn(5, 3, 10)
h0 = torch.randn(2, 3, 20)
output, hn = rnn(input, h0)
```

Reference: https://pytorch.org/docs/stable/generated/torch.nn.RNN.html

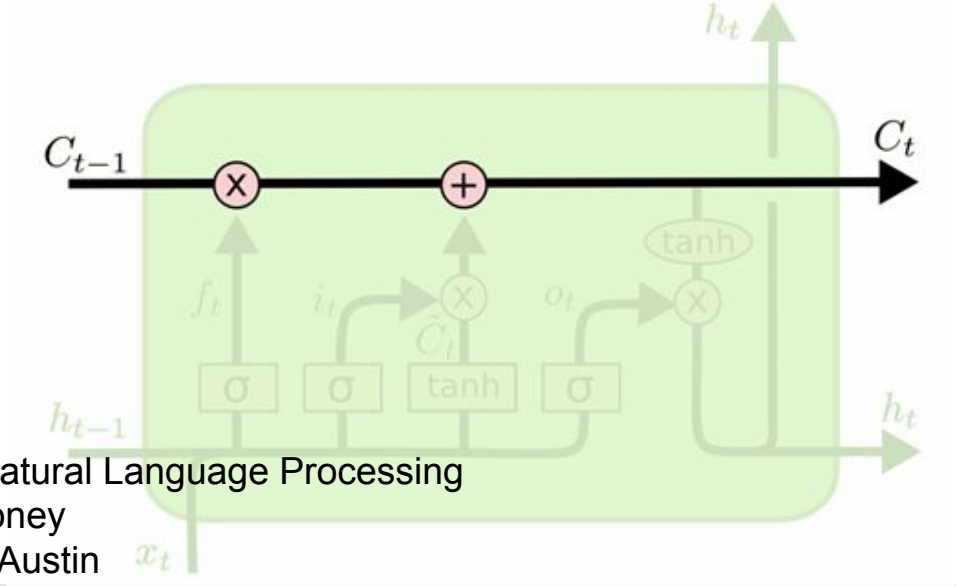
Long Short-Term Memory (LSTM) Model

- LSTM model is a variant of RNN model
- LSTM networks add additional gating units in each memory cell.
  - **Forget gate, Input gate** and **Output gate**
- LSTM prevents vanishing/exploding gradient problem and allows network to retain state information over longer periods of time.



Cell State

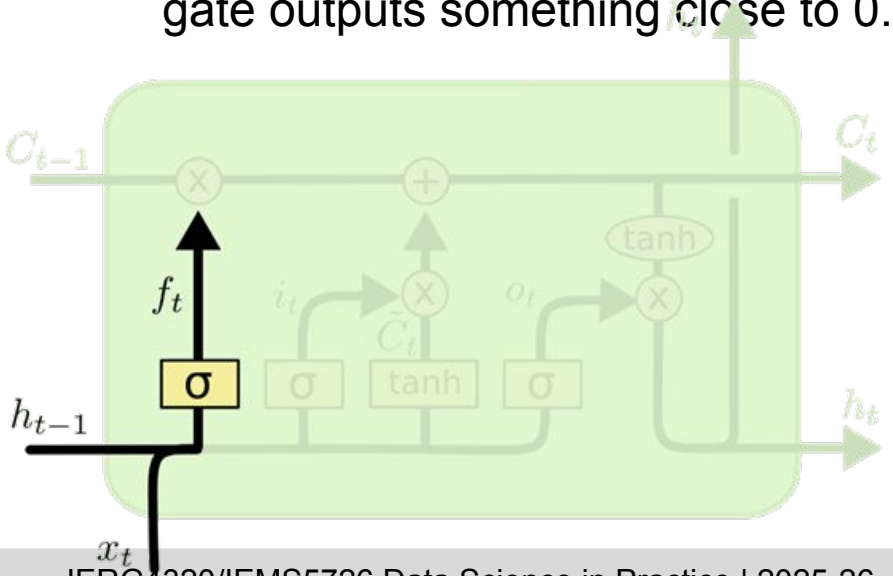
- Cell state is a memory of the LSTM cell.
- Cell state maintains a vector C_t that is the same dimensionality as the hidden state, h_t
- Information can be added or deleted from this state vector via the forget and input gates.



Reference: CS 388: Natural Language Processing
Prof. Raymond J. Mooney
University of Texas at Austin

Forget Gate

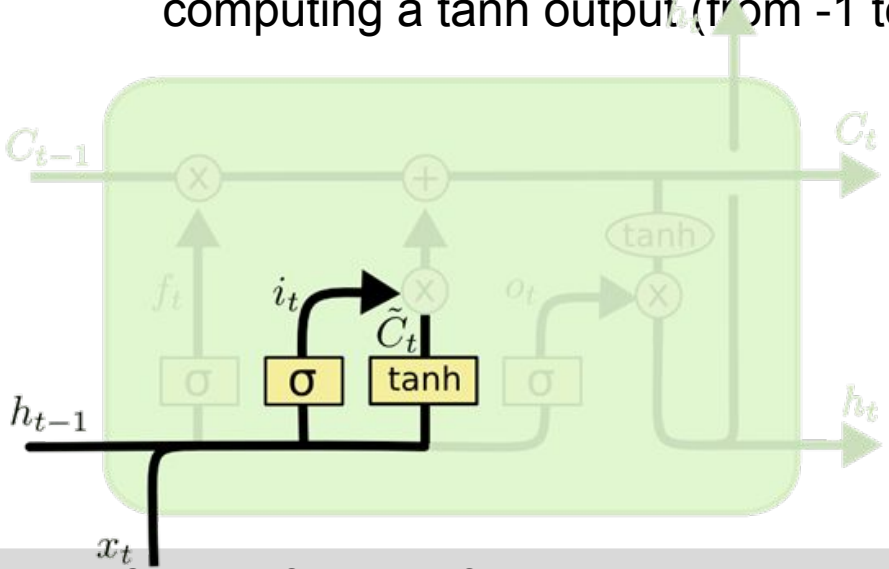
- *What should we throw away from the long-term memory?*
- Forget gate computes a 0-1 value using a logistic sigmoid output function from the input, x_t, and the current hidden state, h_t
- Multiplicatively combined with cell state, "forgetting" information where the gate outputs something close to 0.



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Input Gate

- *What new information should we store in the long-term memory?*
- Input gate determines which entries in the cell state to update by computing 0-1 sigmoid output.
- Input gate determines what amount to add/subtract from these entries by computing a tanh output (from -1 to 1) function of the input and hidden state.

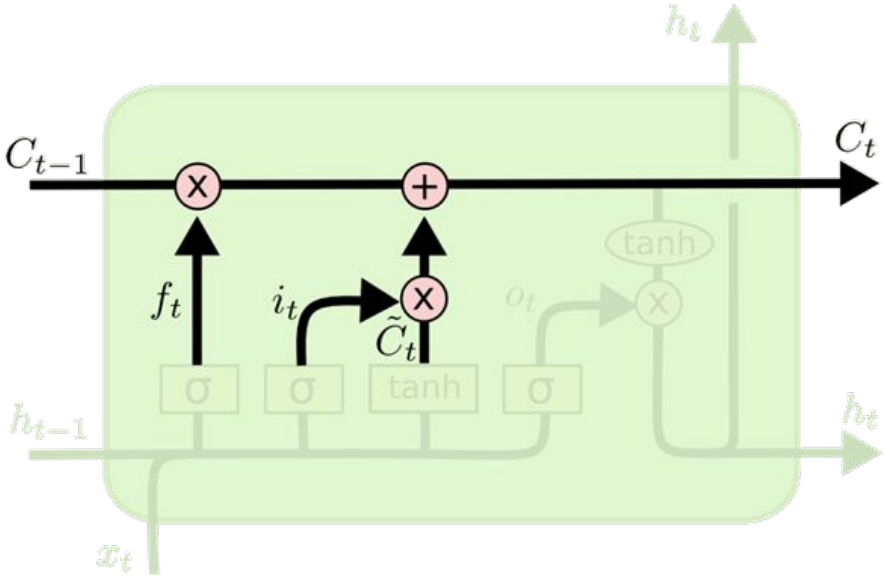


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

Update in the Cell State

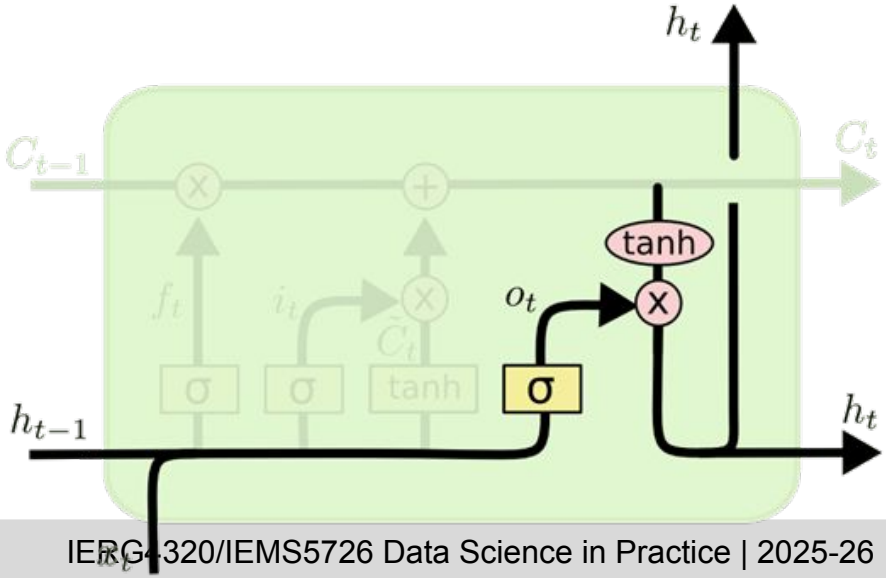
- *Constant Error Carousel*
- Cell state is updated by using componentwise vector multiply to "forget" and vector addition to "input" new information.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate

- *What part of the long-term memory should we output now?*
- Hidden state is updated based on a "filtered" version of the cell state, scaled to –1 to 1 using tanh.
- Output gate computes a sigmoid function of the input and current hidden state to determine which elements of the cell state to "output".



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

More about LSTM

- The gate memory cell regulates the flow of information.
- LSTM is capable of handling stateless information.
  - You can apply fixed-width sliding window on LSTM, so that:
    - The window size = input size = output size.
    - The LSTM input at time step t combines the feature data from both the current window (t) and the output of the previous window (t-1).

A Simple Example of LSTM on PyTorch

```python
import torch
import torch.nn as nn
rnn = nn.LSTM(10, 20, 2)
input = torch.randn(5, 3, 10)
h0 = torch.randn(2, 3, 20)
c0 = torch.randn(2, 3, 20)
output, (hn, cn) = rnn(input, (h0, c0))
```

Reference: https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html

LSTM Approach for Stock Market Index Prediction

1. Data exploration (all data are numerical)
2. Data split using the fixed-width sliding window
3. Model training
   - LSTM Model
   - Regression problem
4. Model assessed using MAE and R^2
5. Interpreting the results with plots

Example: LSTM for Stock Prediction - Model Class

```python
class StockLSTM(nn.Module):
    def __init__(self, input_size=5, hidden_size=50, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True,
            dropout=0.2)
        self.fc1 = nn.Linear(hidden_size, 25)
        self.fc2 = nn.Linear(25, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.dropout(self.relu(self.fc1(out[:, -1, :])))
        out = self.fc2(out)
        return out
```

Model Performance

```
Using device: cpu

Downloading SPY data...
Total data: 2466 days
Train: 2014 days
Test:  452 days

Creating 30-day sliding window...
X_train: (1984, 30, 5)
X_test:  (421, 30, 5)
y_test:  421

Training PyTorch LSTM...
Epoch [10/30], Loss: 0.004088
Epoch [20/30], Loss: 0.003565
Epoch [30/30], Loss: 0.002952

Predicting...
```

```
PYTORCH LSTM RESULTS:
   MAE: 38.4982% (avg error)
   R²:  -2522.486

BACKTEST RESULTS:
Buy & Hold:    +44.2%
PyTorch LSTM:  +0.0%
Edge:          +-44.2%
```

Code: https://colab.research.google.com/drive/1d2uaWcNLMgJlgAHY2qp9AnA4PfCO__ce#scrollTo=du_K0ohgqqRp

Discussions

1. Is the data sufficient enough to cover all the necessary knowledge?
    - News about the company
    - The company performance (Monthly? Weekly? Or even daily?)
    - How to integrate the data?
2. Does the result provide the strategy on buy/sell the stock?
    - Investors are more interested in the investment portfolio.
3. Daily trade has a high investment risk.
    - Investors tend to invest for a longer term.

Illustration: Adjustment based on Company Info

- Use of **Multimodal AI**:
  - Multimodal AI analyzes different inputs and recognizes patterns.
  - Multimodal AI combines different data streams, it can result in more reliable and precise outcomes.
- With the advancement of **transformer**, encoding NL to embeddings is getting more and more popular.

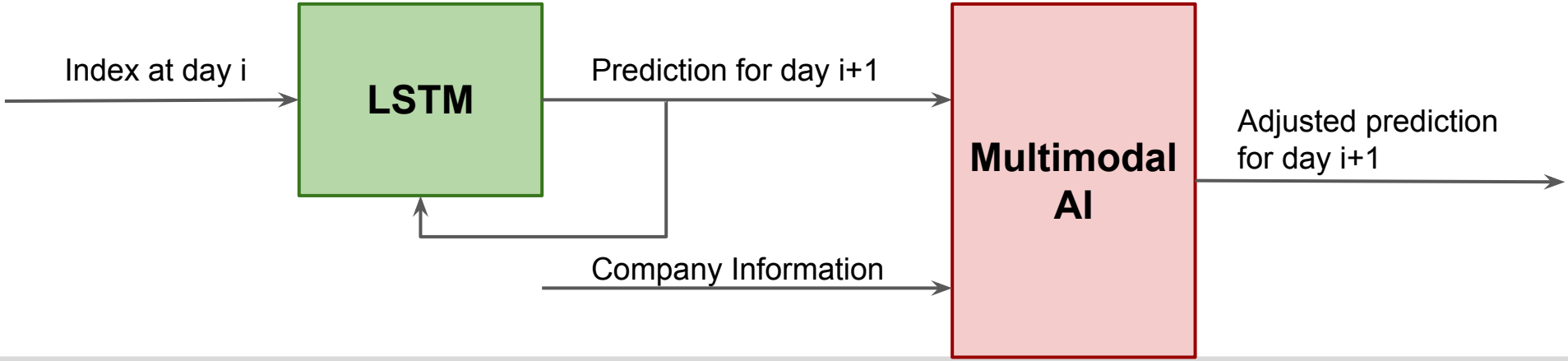
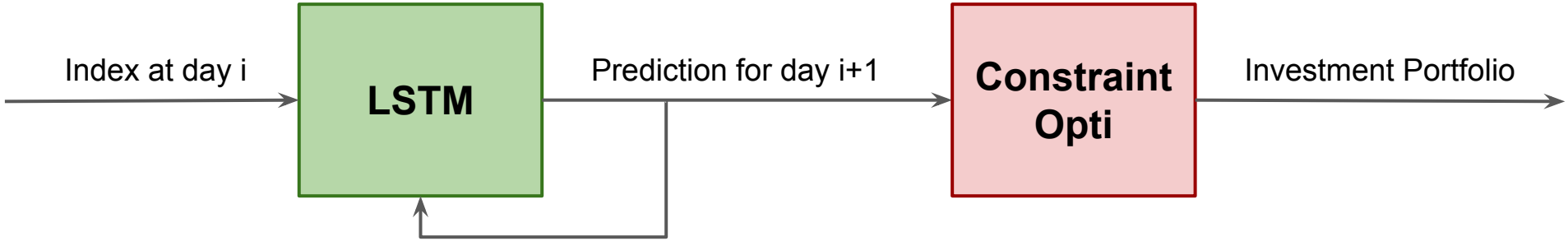


Illustration: Investment Portfolio

- Use of **Constraint Optimization** (or Optimization Model):
  - Max $\sum_k \alpha_k \hat{y}$

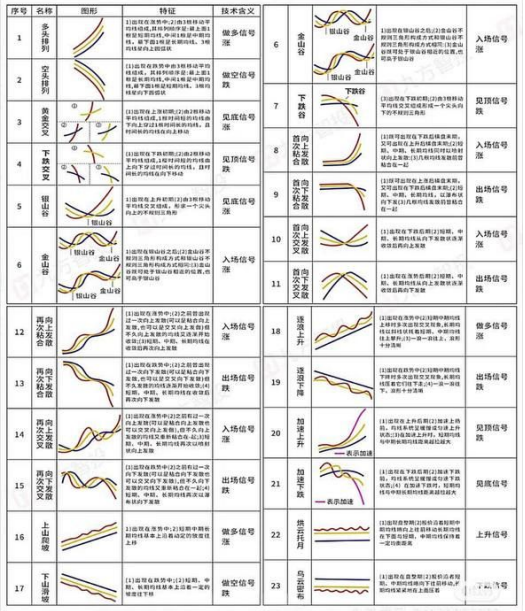
    such that $\sum_k \alpha_k = 1$

    Count of non-zero (α_k) ≤ c, for some parameter c



Reference: https://ieeexplore.ieee.org/document/9121212

Investment Strategy



序号	名称	图形	特征	技术含义
1	多头排列		(1)出现在涨势中;(2)各移动平均线组成,其均线排序是:最上面是短期均线,中间是中期均线,最下面是长期均线。3条均线呈向上的状态	做多信号 涨
2	空头排列		(1)出现在跌势中由3根移动平均线组成,其均线排序是:最上面是长期均线,中间是中期均线,最下面是短期均线。3条均线呈向下的状态	做空信号 跌
3	黄金交叉		(1)出现在上涨初期;(2)由2根移动平均线组成,1根时间较短均线由下向上穿过1根时间较长均线,且时间较长的均线向上移动	见底信号 涨
4	下跌交叉		(1)出现在下跌初期;(2)由2根移动平均线组成,1根时间较短均线由上向下穿过时间长的均线,且时间长的均线向下移动	见顶信号 跌
5	银山谷	银山谷	(1)出现在上升初期;(2)由3根移动平均线交叉组成。形状一个尖头向上的不规则三角形	见底信号 涨
6	金山谷	银山谷 金山谷 银山谷 金山谷	(1)出现在银山谷之后;(2)金山谷不规则三角形构成方式和银山谷不规则三角形构成方式相同;(3)金山谷可能于银山谷稍迟的位置,也可出现于银山谷	入场信号 涨
7	下跌谷	下跌谷	(1)出现在下跌初期;(2)由3根移动平均线交叉组成形成一个尖头向下的不规则三角形	见顶信号 跌
8	首次粘合		(1)有可能存在下跌趋势末期。又可出现在上升趋势末期;(2)短期、中期、长期均线同时以相对较近上涨速(3)几条均线发散后紧贴合在一起	入场信号 涨
9	首次粘合		(1)有可能存在上涨趋势末期。又可出现在下跌趋势末期;(2)短期、中期、长期均线,以及各线向下或以几条均线发散后紧贴合在一起	出场信号 跌
10	首次交叉		(1)出现在下跌趋势;(2)短期、中期、长期均线从向下发散状态开始收敛后再向上发散	入场信号 涨
11	首次交叉		(1)出现在涨势趋势;(2)短期、中期、长期均线从向上发散状态开始收敛后再向下发散	出场信号 跌
12	再次粘合		(1)出现在涨势中;(2)之前曾出现过一次向上发散(可以呈粘合向上发散,也可以呈交叉向上发散)由平头向上发散的均线又选择并粘合呈(2)短期、中期、长期均线出现再次向上发散	入场信号 涨
13	再次粘合		(1)出现在跌势中;(2)之前曾出现过一次向下发散(可以呈粘合向下发散,也可以呈交叉向下发散)由平头发散的均线再次形成发散;(4)短期、中期、长期均线再次出现再次向下发散	出场信号 跌
14	再次交叉		(1)出现在涨势中;(2)之前有过一次向上发散(可以呈粘合向上发散也可以交叉向上发散),但平头向上发散的均线又再形成粘合;(3)短期、中期、长期均线再次以相对较近上发散	入场信号 涨
15	再次交叉		(1)出现在跌势中;(2)之前有过一次向下发散(可以呈粘合向下发散也可以交叉向下发散),但平头向下发散的均线又重新粘合在一起;(4)短期、中期、长期均线再次以重新状的下发散	出场信号 跌
16	上山回吸		(1)出现在涨势中;(2)短期中期均线均呈重上沿易动区间第度往上升	做多信号 涨
17	下山滑坡		(1)出现在跌势中;(2)短期、中期、长期均线基本上沿着一定角度往下滑	做空信号 跌
18	迷途上升		(1)出现在涨势中;(2)短期中期均线上呈相多次出现交叉状态,长期均线以相对较低角度、平期均线往上呈平行或一条一条往上,且形平行流畅	做多信号 涨
19	迷途下降		(1)出现在跌势中;(2)短期中期均线下呈相多次出现交叉状态,长期均线呈相对较低下呈;(4)一条一条往下,且形平行流畅	出场信号 跌
20	加速上升	表示加速	(1)出现在上升后期;(2)呈进上升状态;(3)在加速上升中,短期均线与中期长期均线呈喇叭状张大	见顶信号 跌
21	加速下跌	表示加速	(1)出现在下跌后期;(2)呈进下跌状态;(4)在加速下跌中,短期均线与中期长期均线呈喇叭状张大	见底信号
22	烘云托月		(1)出现在整理;(2)相对沿着短期中期均线横向上移且移动长期均线与平呈均线,中期均线保持着一定均衡度来	上升信号
23	乌云密布		(1)出现在盘整期;(2)相对沿着短期、中期均线横向下移且移动,长期均线呈相向上横压着	[illegible]信号

Image credit: Futu Bull

An **investment strategy** is a systematic plan or set of rules designed to guide an investor's decision-making process to achieve specific financial goals. It is like a roadmap for your money, helping you decide what to buy, when to buy it, and when to sell it.

Components of an Investment Strategy

1. **Financial Goals**: What are you investing for? (e.g., retirement in 30 years, a down payment on a house in 5 years, a child's education in 15 years).
2. **Time Horizon**: How long can you keep your money invested? This is directly tied to your goals and is a critical factor in determining your risk tolerance.
3. **Risk Tolerance**: How much volatility (ups and downs in your portfolio's value) are you comfortable with? This is both a financial and emotional question.
4. **Asset Allocation**: The most important decision. This is how you divide your investment funds among major asset classes like:
   - Stocks (Equities): For growth; higher risk.
   - Bonds (Fixed Income): For income and stability; lower risk.
   - Cash & Equivalents: For liquidity and safety; lowest risk/return.
   - Alternative Investments: Real estate, commodities, etc.
5. **Specific Investment Selection**: Once you have an asset allocation, you choose the specific stocks, bonds, or funds that fit within that plan.

Why Investment Strategy Important?

1. **Provides Discipline**: It removes emotion from investing. When markets are volatile, a strategy helps you stick to the plan instead of making panic-driven decisions like selling at a market bottom.
2. **Manages Risk**: By defining your asset allocation, you systematically control the level of risk in your portfolio.
3. **Improves Consistency**: It creates a repeatable process, preventing you from chasing the latest "hot" trend, which is often a recipe for losses.
4. **Measures Performance**: It gives you a benchmark against which to measure your portfolio's performance. Without a strategy, you have no way of knowing if your investments are successful.

Non-ML Investment Strategies

- Based on **Activity Level**

1. **Active Investing**: The investor (or a fund manager) actively tries to “beat the market” by frequently buying and selling securities based on research, market timing, and forecasts.
   - Examples: Growth investing, Value investing.
2. **Passive Investing**: The investor aims to match market returns, not beat them. This is typically done by buying and holding a diversified portfolio that tracks a market index (like the S&P 500).
   - Examples: Index fund investing, Buy-and-Hold strategy.

Non-ML Investment Strategies

- Based on **Style and Philosophy**

1. **Value Investing**: Buying stocks that appear undervalued relative to their intrinsic worth (like buying a dollar for fifty cents). Example: Warren Buffett.
2. **Growth Investing**: Buying stocks of companies that are expected to grow at an above-average rate compared to the market, even if their current stock price seems high.
3. **Income Investing**: Focusing on generating a steady income from investments through dividends (stocks) or interest (bonds).
4. **Dividend Growth Investing**: A hybrid strategy focused on companies that not only pay dividends but consistently increase them year after year.
5. **Dollar-Cost Averaging (DCA)**: A technique of investing a fixed amount of money at regular intervals (e.g., monthly), regardless of the share price. This reduces the risk of investing a large amount at the wrong time.

Non-ML Investment Strategies

- Based on **Risk and Goals**

1. **Conservative Strategy**: Prioritizes capital preservation. High allocation to bonds and cash. Suitable for short-term horizons or low-risk tolerance.
2. **Moderate Strategy**: Seeks a balance between growth and income. A mix of stocks and bonds (e.g., 60/40 portfolio).
3. **Aggressive Strategy**: Aims for maximum growth. High allocation to stocks. Suitable for long time horizons and high-risk tolerance.

Non-ML ways to develop Investment Strategy

1. **Define Your Goals**: Be specific (e.g., "I need $1.5 million to retire at age 65").
2. **Assess Your Time Horizon**: When will you need the money?
3. **Determine Your Risk Tolerance**: Be honest about how you would react to a 20% market drop.
4. **Establish Your Asset Allocation**: Based on the steps above, decide your stock/bond/cash mix.
5. **Select Your Investments**: Choose specific index funds, ETFs, or stocks that fit your allocation.
6. **Implement and Monitor**: Execute your plan and review it periodically (e.g., annually) to ensure it's still on track. Rebalance if necessary to maintain your target asset allocation.

How ML Approaches linking to Investment Strategy

- Quantitative finance is at the intersection of computer science, statistics and economics. For example:

1. A bond issued by a government pays coupons at 5% every 6 months for the next 3 years. What should be its current price for every dollar of the principal?
2. An option gives the option holder the right to buy 1000 shares of Facebook at USD$280 per share one year from now? What should the current option price be?
3. You have $100000 to invest in shares of SIA, Singtel and DBS for one year. How should the investor allocate the money so that the investment has minimum risk and with an expected return of at least 5%?

Reference: https://www.math.nus.edu.sg/wp-content/uploads/sites/4/2023/06/20230227-Slides-THH-QF.pdf

Example 1: Directional Forecasting

- Directional forecasting is a type of forecasting that prioritizes predicting the future direction of a trend rather than its precise numerical value. The primary question it answers is: “Will the metric go Up, go Down, or stay Flat?” rather than “What will the exact value be?”
- **Strategy**: Binary prediction: Buy if 1, Sell if 0.
- **Data**: Past prices, volumes, macro data
- **Target**: when Price(t+1) > Price(t), result = 1; else result = 0

Example 1: Directional Forecasting

1. Download the data from yfinance
2. Extract or create the directional targets
3. Prepare fixed width sliding window for training and testing
4. Prepare the LSTM model and train it
5. Test the model and visualize the results

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I?usp=sharing

Model Performance

```
Downloading SPY data...
Total data: 2466 days
Clean data: 2465 days
UP days: 1345/2465 (54.6%)
Train: 2014 days (2015-2022)
Test:  451 days (2023-2024)

Creating 30-day windows...
X_train: (1984, 30, 5), y_train: (1984,)
X_test:  (421, 30, 5), y_test: (421,)

Model Summary:
DirectionalLSTM(
  (lstm): LSTM(5, 50, num_layers=2, batch_first=True, dropout=0.2)
  (fc1): Linear(in_features=50, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=1, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
  (sigmoid): Sigmoid()
)
```

Model Performance

```
Training LSTM (30 epochs)...
Epoch [10/30], Loss: 0.6902, Acc: 53.9%
Epoch [20/30], Loss: 0.6898, Acc: 53.7%
Epoch [30/30], Loss: 0.6896, Acc: 53.9%
Testing model...
Test Accuracy: 57.5%

Classification Report:
              precision    recall  f1-score   support
        DOWN       0.00      0.00      0.00       179
          UP       0.57      1.00      0.73       242
    accuracy                           0.57       421
   macro avg       0.29      0.50      0.37       421
weighted avg       0.33      0.57      0.42       421

Backtest Results (2023-2024):
Buy & Hold:  +44.2%
Strategy:    +44.2%
Edge:        ++0.0%
```

Example 2: Return Prediction

- Return prediction is the process of forecasting the future financial return of an asset over a specific holding period. Unlike simple price forecasting, return prediction specifically focuses on the percentage gain or loss that an investment is expected to generate.
- The investor asks “What is the expected rate of return I will earn on this investment over the next day, next month or next year?”
- **Strategy**: Buy assets with highest predicted return.
- **Data**: Past prices, volumes, technical indicators
- **Target**: Actual return over next N days, e.g., 7-day return

Example 2: Return Prediction

1. Download the data from yfinance
2. Extract or create the 7-day return targets
3. Prepare fixed width sliding window for training and testing
4. Prepare the LSTM model and train it
5. Test the model and visualize the results

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=rtUXWLI8IEBF

Model Performance

```
Downloading SPY data...
Total data: 2466 days
Clean data: 2459 days
7-day returns: Mean 0.38%, Std 2.65%
Train: 2014 days (2015-2022)
Test:  445 days (2023-2024)

Creating 30-day windows...
X_train: (1977, 30, 5), y_train: (1977,)
X_test:  (408, 30, 5), y_test: (408,)

Model Summary:
ReturnLSTM(
  (lstm): LSTM(5, 50, num_layers=2, batch_first=True, dropout=0.2)
  (fc1): Linear(in_features=50, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=1, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Model Performance

```
Training LSTM (30 epochs)...
Epoch [10/30], Loss: 7.6689, MAE: 1.89%
Epoch [20/30], Loss: 7.7046, MAE: 1.90%
Epoch [30/30], Loss: 7.6477, MAE: 1.89%

Testing model...

Test Results:
MAE:           1.68% (avg error)
R²:            -0.056
Directional Acc: 60.5%

Backtest Results (Weekly, 2023-2024):
Buy & Hold:  +1250.3%
Strategy:    +645.9%
Edge:        +-604.4%
```

Example 3: Volatility Prediction

- Volatility prediction is the process of forecasting the future variability or dispersion of an asset's returns over a specific period. Volatility prediction is all about forecasting the degree of uncertainty or risk.
- The investor asks "How much is this asset's price likely to fluctuate in the future?"
- **Strategy**: Risk management, adjust position sizes.
- **Data**: Historical volatility, sentiment
- **Target**: Realized volatility over next N days

Example 3: Volatility Prediction

1. Download the data from yfinance
2. Extract or create the volatility targets
   - Calculate the weekly return
   - Calculate the standard deviation of the return
   - Annualize the volatility by scaling the standard deviation by the square root of the number of trading days (roughly 252 days per year)
3. Prepare fixed width sliding window for training and testing
4. Prepare the LSTM model and train it
5. Test the model and visualize the results

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=V-kvc3SlpvcJ

Model Performance

```
Downloading SPY data...
Total data: 2466 days
Clean data: 2459 days
7-day returns: Mean 0.38%, Std 2.65%
Train: 2014 days (2015-2022)
Test:  445 days (2023-2024)

Creating 30-day windows...
X_train: (1977, 30, 5), y_train: (1977,)
X_test:  (408, 30, 5), y_test: (408,)

Model Summary:
ReturnLSTM(
  (lstm): LSTM(5, 50, num_layers=2, batch_first=True, dropout=0.2)
  (fc1): Linear(in_features=50, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=1, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Model Performance

```
Training LSTM (30 epochs)...
Epoch [10/30], Loss: 7.6689, MAE: 1.89%
Epoch [20/30], Loss: 7.7046, MAE: 1.90%
Epoch [30/30], Loss: 7.6477, MAE: 1.89%

Testing model...

Test Results:
MAE:           1.68% (avg error)
R²:            -0.056
Directional Acc: 60.5%

Backtest Results (Weekly, 2023-2024):
Buy & Hold:  +1250.3%
Strategy:    +645.9%
Edge:        +-604.4%
```

Example 4: Portfolio Optimization

- Portfolio optimization is the quantitative process of selecting the best possible investment portfolio from a universe of assets, based on a defined objective and a set of constraints. The “best” portfolio is one that offers the highest expected return for a given level of risk, or conversely, the lowest risk for a given level of expected return.
- The investor asks “How should I distribute my money among different investments to achieve my financial goals while staying within my risk tolerance?”
- **Strategy**: Use ML predictions as inputs to some optimizers.
- **Data**: Predicted returns, covariances for all assets
- **Target**: Optimal weight for each asset in portfolio

Example 4: Portfolio Optimization

1. Download the data from yfinance
2. Extract or create the portfolio targets for 4 assets
3. Prepare fixed width sliding window for training and testing
4. Prepare the LSTM model + Mean-Variance optimization (MVO) and train it
    - The best portfolio includes maximum expected return for a given level of risk and minimum risk for a given level of expected return
    - Initialize the portfolio weights
    - Optimize the weights based on return and/or variance
5. Test the model and visualize the results

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=10MeZVyz87aX

Model Performance

```
Downloading 4-asset portfolio data...
Total data: 2466 days
Assets: ['SPY', 'QQQ', 'IWM', 'EEM']
Clean data: 2452 days
7-day returns: Mean 0.35%
Train: 2007 days (2015-2022)
Test:  445 days (2023-2024)

Creating 30-day windows...
X_train: (1970, 30, 4), y_train: (1970,)
X_test:  (408, 30, 4), y_test: (408,)

Model Summary:
PortfolioLSTM(
  (lstm): LSTM(4, 50, num_layers=2, batch_first=True, dropout=0.2)
  (fc1): Linear(in_features=50, out_features=25, bias=True)
  (fc2): Linear(in_features=25, out_features=1, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Model Performance

```
Training LSTM (30 epochs)...
Epoch [10/30], Loss: 9.1011
Epoch [20/30], Loss: 9.0803
Epoch [30/30], Loss: 8.8778

Testing portfolio optimization...
Prediction MAE: 2.01%
Weekly returns available: 438 weeks

PORTFOLIO OPTIMIZATION RESULTS (2023-2024):
Optimized Portfolio: +122.7%
Equal Weight:        +39.9%
S&P 500:             +47.9%
EDGE vs Equal:       ++82.8%
Sharpe Ratio:        3.58 vs 1.75
```

Financial Metrics - Evaluating ML models for Business

1. **Total Return**: The overall percentage change in the value of an investment over a specific period
   - Total Return = (Ending Value - Beginning Value + Dividends) / Beginning Value ✖ 100%
2. **Sharpe Ratio**: Used for risk-adjusted return. Measures how much excess return you are receiving for the extra volatility you endure by holding a riskier asset.
   - Sharpe Ratio = (Portfolio Return - Risk-Free Rate) / Standard Deviation of Portfolio Returns
3. **Maximum Drawdown (MDD)**: The maximum observed loss from a peak to a trough of a portfolio, before a new peak is attained.
   - MDD = (Trough Value - Peak Value) / Peak Value
4. **Sortino Ratio**: A variation of the Sharpe Ratio that differentiates harmful volatility (downside risk) from total overall volatility.
   - Sortino Ratio = (Portfolio Return - Target Return) / Downside Deviation

RL Approach for Investment Strategy

- An agent (the AI trader) learns to make profitable decisions by interacting with the environment (the stock market).

1. **Agent**: The algorithm that makes trading decisions.
2. **Environment**: The stock market (prices, news, economic data).
3. **State (S)**: The current situation (e.g., current portfolio holdings, past 30 days of prices, technical indicators, news sentiment).
4. **Action (A)**: What the agent can do (e.g., Buy, Sell, Hold a stock, or manage portfolio weights).
5. **Reward (R)**: The profit or loss from an action (e.g., change in portfolio value, or a risk-adjusted return like the Sharpe Ratio).

Continuous Feedback Loop

1. **Observe State (s_t)**: The agent looks at the current market data (e.g., “Stock A is rising, my portfolio is up 2%”).
2. **Take Action (a_t)**: Based on its current policy, the agent decides what to do (e.g., Sell / hold / buy Stock A).
3. **Receive Reward (r_t)**: The environment provides feedback to the agent (e.g., the agent sold just before a price drop, receiving a positive reward. Alternatively, if the price had soared, receiving a negative reward).
4. **Observe New State (s_{t+1})**: The market moves to a new state, i.e the next sliding window.
5. **Learn and Update**: The agent uses this experience **(s_t, a_t, r_t, s_{t+1})** to update its policy, making successful actions more likely in similar future states.

Continuous Feedback Loop = Training Environment

```
class TradingEnv:
    def __init__(self, df, window=30, initial_balance=10000):
        self.df = df.reset_index(drop=True)
        self.window = window
        self.initial_balance = initial_balance
        self.current_step = window
        self.balance = initial_balance
        self.shares_held = 0
        self.total_shares_sold = 0
        self.total_profit = 0
        self.trades = []
        self.portfolio_history = [initial_balance]
```

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=5uaEAIl5yY6j

Continuous Feedback Loop = Training Environment

```
    def reset(self):
        self.current_step = self.window
        self.balance = self.initial_balance
        self.shares_held = 0
        self.total_shares_sold = 0
        self.total_profit = 0
        self.trades = []
        self.portfolio_history = [self.initial_balance]
        return self._get_state()

    def _get_state(self):
        start = self.current_step - self.window
        end = self.current_step
        prices = self.df['Close'].iloc[start:end].values
        state = np.concatenate([prices, [self.balance/10000, self.shares_held/100]])
        return state[:31]
```

Continuous Feedback Loop = Training Environment

```python
    def step(self, action):
        current_price = self.df['Close'].iloc[self.current_step]
        reward = 0
        if action == 1 and self.balance > current_price:  # BUY
            shares_bought = int(self.balance // current_price)
            cost = shares_bought * current_price
            if shares_bought > 0:
                self.shares_held += shares_bought
                self.balance -= cost
                self.trades.append(('BUY', self.current_step, shares_bought,
                    current_price))
        elif action == 2 and self.shares_held > 0:  # SELL
            revenue = self.shares_held * current_price
            self.balance += revenue
            self.total_shares_sold += self.shares_held
            self.shares_held = 0
            self.trades.append(('SELL', self.current_step, self.shares_held,
                current_price))
```

Continuous Feedback Loop = Training Environment

```
        self.current_step += 1
        done = self.current_step >= len(self.df) - 1
        portfolio_value = self.balance + self.shares_held * self.df['Close'].iloc[-1]
        self.portfolio_history.append(portfolio_value)
        reward = (portfolio_value - self.initial_balance) * 0.01
        next_state = self._get_state()
        return next_state, reward, done, {'portfolio_value': portfolio_value}
```

Deep Q-Networks (DQN)

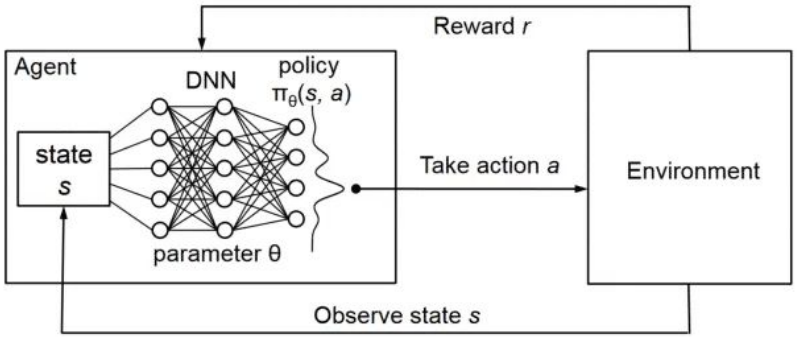


Image credit: Dilith Jayakody

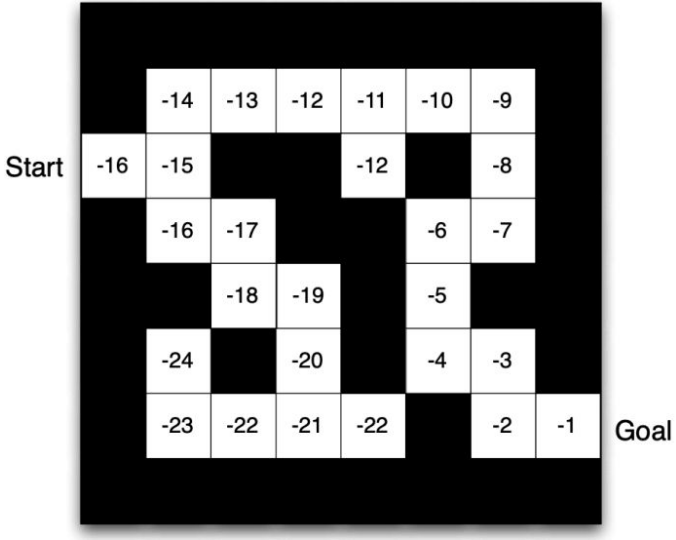
Deep Q-Networks (DQN)

- DQN is a groundbreaking AI algorithm that **combines reinforcement learning with deep learning** in 2013.
- First application: Learned to play Atari games from the raw pixels of the screen, and surpassed human level.
- DQN improves from the Q-Learning, and is belong to value-based approach.
  - They choose the action with the highest predicted future reward.

Value-Based Methods

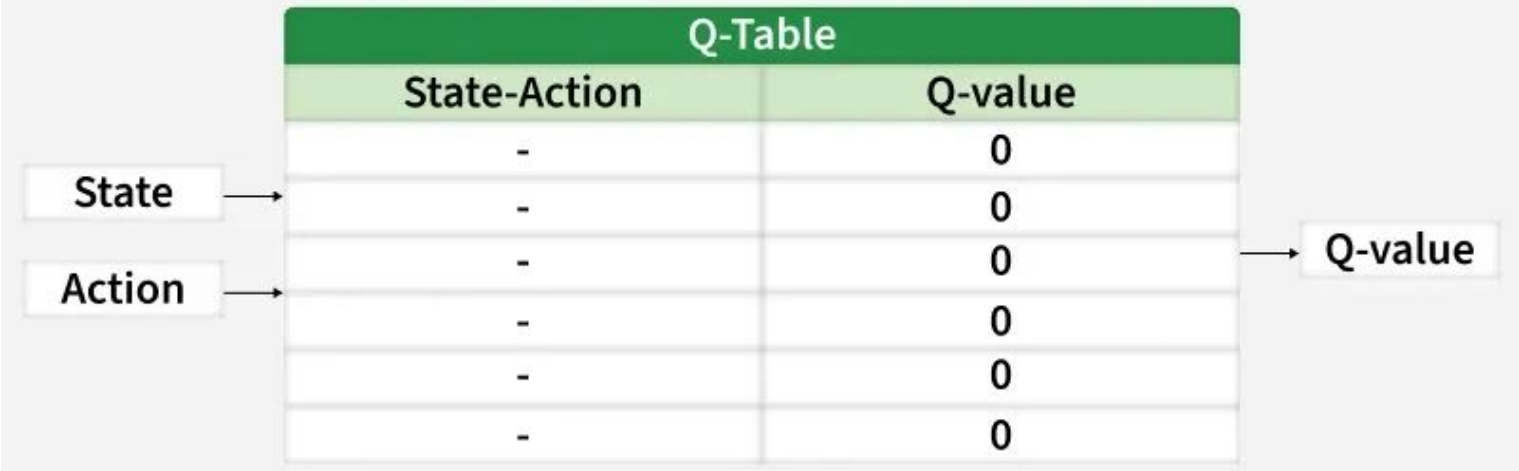
- The goal is to learn the optimal value function (V(s) or Q(s,a)). The policy is then derived directly from the value function.

Value-based:



Q-Learning Agent

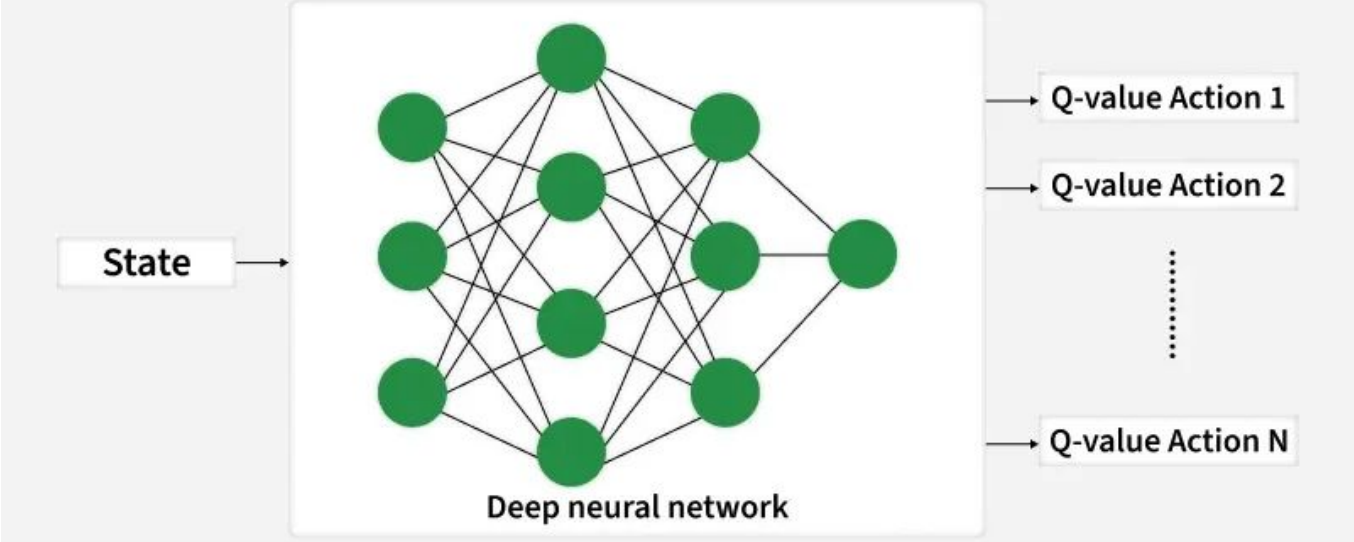
- Q-Learning is a popular model-free reinforcement learning algorithm that helps an agent learn how to make the best decisions by interacting with its environment.
- Q-Learning agent learns purely from experience by trying different actions and seeing their results.



Reference: https://www.geeksforgeeks.org/deep-learning/deep-q-learning/

Deep Q-Networks (DQN) Agent

- Deep Q-Learning is a method that uses deep learning to help machines make decisions in complicated situations. It is especially useful in environments where the number of possible situations called states is very large like in video games or robotics.



Reference: https://www.geeksforgeeks.org/deep-learning/deep-q-learning/

Key Challenges Addressed by Deep Q-Learning

1. **High-Dimensional State Spaces**: Traditional Q-Learning uses a table to store values but this becomes impossible when there are too many situations. Neural networks can understand and work with many different situations at once so they are better for complex problems.
2. **Continuous Input Data**: Real-world problems often have continuous data like video images. Neural networks are good at handling this kind of information.
3. **Scalability**: Deep learning helps Q-Learning grow and handle bigger, harder tasks that regular Q-Learning couldn't solve before.

Reference: https://www.geeksforgeeks.org/deep-learning/deep-q-learning/

A Simple DQN

```python
class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, n_actions)

    # Called with either one element to determine next action, or a batch
    # during optimization. Returns tensor([[left0exp,right0exp]...]).
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        return self.layer3(x)


```

Reference: https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

DQN for Investment Strategy

```python
class DQN(nn.Module):

    def __init__(self, state_size=31, action_size=3):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.fc3(x)
```

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=5uaEAIl5yY6j

Training Process

1. **Initialization**:
   - Initialize the replay buffer, main network (θ) and target network (θ^-).
   - Set hyperparameters such as learning rate (α), discount factor (γ) and exploration rate (ϵ).
2. **Exploration vs. Exploitation**: Use an ϵ-greedy policy to balance exploration and exploitation:
   - With probability ϵ, select a random action to explore.
   - Otherwise, choose the action with the highest Q-value according to the current network.
3. **Experience Collection**: Interact with the environment, collect experiences (s,a,r,s') and store them in the replay buffer.

Training Process

4. **Training Updates**:
   - Sample a mini-batch of experiences from the replay buffer.
   - Compute the target Q-values using the target network.
   - Update the main network by minimizing the loss function using gradient descent.
5. **Target Network Update**: Periodically copy the weights of the main network to the target network to ensure stability.
6. **Decay Exploration Rate**: Gradually decrease ϵ over time to shift from exploration to exploitation.

Replay Memory

```python
class ReplayMemory:
    def __init__(self, capacity=5000):
        self.memory = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        return random.sample(self.memory, batch_size)

    def __len__(self):
        return len(self.memory)
```

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=5uaEAIl5yY6j

DQN Agent for Investment Strategy

```python
class DQNAgent:

   def __init__(self, state_size=31, action_size=3):  # Step 1: Initialization
        self.state_size = state_size
        self.action_size = action_size
        self.memory = ReplayMemory(5000)
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.model = DQN(state_size, action_size).to(device)
        self.target_model = DQN(state_size, action_size).to(device)
        self.optimizer = optim.Adam(self.model.parameters(), lr=self.learning_rate)
        self.criterion = nn.MSELoss()
        self.update_target_model()

```

Code: https://colab.research.google.com/drive/1333GaJidUISxpKdmBJnno1XyozxEg58I#scrollTo=5uaEAIl5yY6j

DQN Agent for Investment Strategy

```
    def update_target_model(self):   # Step 5: Target network update
        self.target_model.load_state_dict(self.model.state_dict())
    # Remark: Step 3 (Experience collection) is related to the environment, not DQN
    def act(self, state):
        if np.random.rand() <= self.epsilon:  # Step 2: Exploration vs Exploitation
            return random.randrange(self.action_size)

        state = torch.FloatTensor([np.pad(state, (0, 31-len(state)))]) \
                .to(device)
        q_values = self.model(state)
        return np.argmax(q_values.cpu().data.numpy())

    def replay(self, batch_size):
        if len(self.memory) < batch_size: return

        minibatch = self.memory.sample(batch_size)  # Step 4: Training updates
        states, actions, rewards, next_states, dones = zip(*minibatch)
        states = torch.FloatTensor([np.pad(s, (0, 31-len(s))) for s in
            states]).to(device)
```

DQN Agent for Investment Strategy

```
        actions = torch.LongTensor(actions).to(device)
        rewards = torch.FloatTensor(rewards).to(device)
        next_states = torch.FloatTensor([np.pad(s, (0, 31-len(s))) for s in
            next_states]).to(device)
        dones = torch.BoolTensor(dones).to(device)

        current_q = self.model(states).gather(1, actions.unsqueeze(1))
        next_q = self.target_model(next_states).max(1)[0].detach()
        target_q = rewards + (self.gamma * next_q * ~dones)

        loss = self.criterion(current_q, target_q.unsqueeze(1))

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        if self.epsilon > self.epsilon_min:  # Step 6: Decay exploration rate
            self.epsilon *= self.epsilon_decay

```

Model Performance

```
Downloading SPY data...
Total data: 2466 days

Initializing DQN Agent...

TRAINING DQN AGENT...
Episode 0, Portfolio: $12413, Epsilon: 0.995
Episode 20, Portfolio: $11456, Epsilon: 0.900
Episode 40, Portfolio: $10753, Epsilon: 0.814
Episode 60, Portfolio: $16395, Epsilon: 0.737
Episode 80, Portfolio: $15717, Epsilon: 0.666
```

Model Performance

```
TESTING DQN AGENT...
DQN RESULTS:
DQN Final:    $15051
Buy & Hold:   $15651
DQN Return:   +50.5%
Buy & Hold:   +56.5%
EDGE:         +-6

======================================================================
DQN REINFORCEMENT LEARNING AGENT
======================================================================
Final Portfolio: $15051
EDGE OVER MARKET: +-6
======================================================================
```

Major Limitations for DQN

1. **Overestimation Bias**: The DQN algorithm has a tendency to be overly optimistic. The network then learns to chase the maximum Q-values, leading to a high risk investment.
2. **Discrete Action Space**: In the investment of stock market index, it is so happened that the action space is discrete, i.e. {buy, sell, hold}. However, not all action space are discrete, e.g. motion of robots.
3. **Insufficient Information**: In the classic reinforcement learning, the action of the agent can affect the environment (e.g. Atari game, board game). However, from the agent perspective, the stock market index is the result of lots of uncontrolled factors (e.g. company fundamentals, macroeconomic data and market sentiment).

Summary

- We gone through (i) **recommender system**, (ii) **stock market index prediction** and (iii) **investment strategy**
- Techniques for splitting the **time series data**
- **Daily trade prediction and DQN are high risk**, focus more on long term investment
- Think about whether the data (financial data) is sufficient for index prediction or not
- Highlight of some deep learning models, **DeepCoNN**, **Wide and Deep Model**, **Graph Convolutional Network (GCN)**, **Long Short-Term Memory (LSTM)** and **Deep Q-Network (DQN)**.